

xv6 及 Labs 课程项目文档

源码链接: <https://github.com/Lanqi2345/xv6-labs-2020.git>

xv6 及 Labs 课程项目文档

环境配置

一、安装 WSL 2

1. 启用所需Windows功能
2. 安装Ubuntu

二、配置XV6系统环境

1. 前置准备
2. 安装XV6

三、关于 git 的使用

1. 建立自己的仓库
2. 修改本地代码的“关联”
3. 推送

Lab: Xv6 and Unix utilities

Boot xv6
sleep
pingpong
primes
find
xargs
Lab总结

Lab: system calls

System call tracing
Sysinfo
Lab总结

Lab: page tables

Print a page table
A kernel page table per process
Simplify
Lab总结

Lab: traps

RISC-V assembly
Backtrace
Alarm
Lab总结

Lab: xv6 lazy page allocation

Eliminate allocation from sbrk()
Lazy allocation
Lazytests and Usertests
Lab总结

Lab: Copy-on-Write Fork for xv6

Implement copy-on write
Lab总结

Lab: Multithreading

Uthread: switching between threads
Using threads
Barrier
Lab总结

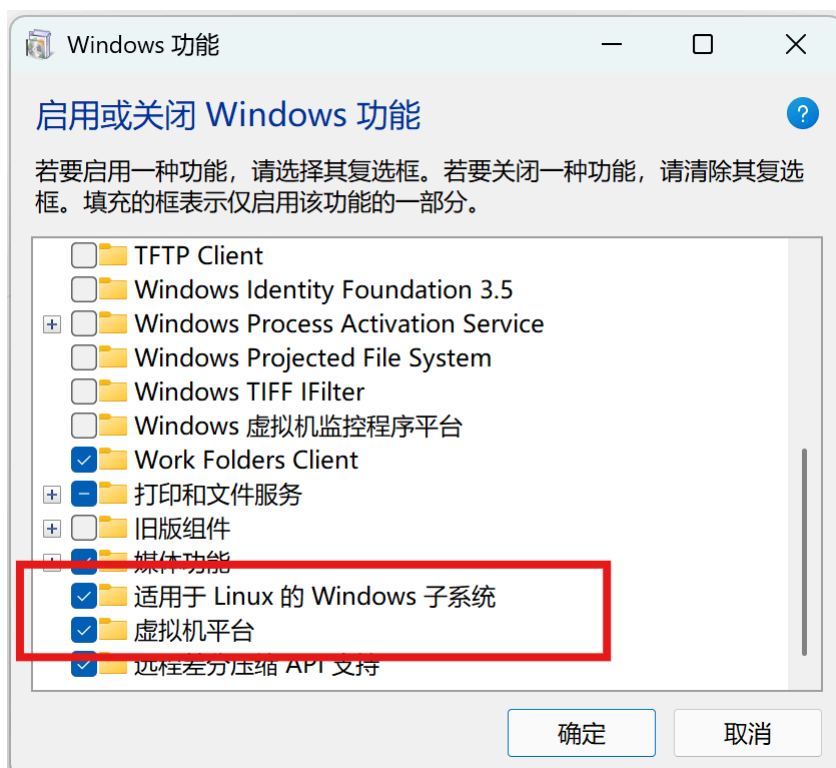
Lab: locks
Memory allocator
Buffer cache
Lab总结
Lab: file system
Large files
Symbolic links
Lab总结
Lab: mmap
Lab: mmap
Lab总结
Lab: networking
Lab: networking
Lab总结
总结

环境配置

个人电脑的操作系统为Windows 11

一、安装 WSL 2

1. 启用所需Windows功能



确保勾选“虚拟机平台”和“适用于Linux的Windows子系统”

2. 安装Ubuntu

查看可安装的Linux发行版

```
1 | wsl --list --online
```

```
PS C:\Users\兰淇\Desktop> wsl --list --online
以下是可安装的有效分发的列表。
使用“wsl.exe --install <Distro>”安装。

NAME                                FRIENDLY NAME
Ubuntu                              Ubuntu
Ubuntu-26.04                        Ubuntu 26.04 LTS
Ubuntu-24.04                        Ubuntu 24.04 LTS
Ubuntu-22.04                        Ubuntu 22.04 LTS
openSUSE-Tumbleweed                 openSUSE Tumbleweed
openSUSE-Leap-16.0                  openSUSE Leap 16.0
SUSE-Linux-Enterprise-15-SP7        SUSE Linux Enterprise 15 SP7
SUSE-Linux-Enterprise-16.0          SUSE Linux Enterprise 16.0
kali-linux                          Kali Linux Rolling
Debian                              Debian GNU/Linux
AlmaLinux-8                         AlmaLinux OS 8
AlmaLinux-9                         AlmaLinux OS 9
AlmaLinux-Kitten-10                 AlmaLinux OS Kitten 10
AlmaLinux-10                        AlmaLinux OS 10
archlinux                           Arch Linux
FedoraLinux-44                      Fedora Linux 44
FedoraLinux-43                      Fedora Linux 43
eLxR                                eLxR 12.12.0.0 GNU/Linux
OracleLinux_7_9                     Oracle Linux 7.9
OracleLinux_8_10                    Oracle Linux 8.10
OracleLinux_9_5                     Oracle Linux 9.5
SUSE-Linux-Enterprise-15-SP6        SUSE Linux Enterprise 15 SP6
```

安装

```
1 | wsl.exe --install Ubuntu
```

检查版本

```
1 | wsl -l -v
```

```
PS C:\Users\兰淇\Desktop> wsl -l -v
NAME                STATE      VERSION
* Ubuntu            Running    2
docker-desktop      Stopped    2
```

初次打开，进行设置用户名和密码设置。

二、配置XV6系统环境

1. 前置准备

参考博客[Mit6.s081环境配置踩坑之旅WSL2+VScode mit6s081-CSDN博客](#)进行换源，提升网络速度。

输入：

```
1 | sudo vim /etc/apt/sources.list
```

进入编辑界面，按“gg”定位到所有行的开头，按“dG”删掉所有内容。按键盘上的“INSERT”键进行编辑，复制中科大的源。

```
1 deb https://mirrors.ustc.edu.cn/ubuntu/ focal main restricted universe multiverse
2 deb-src https://mirrors.ustc.edu.cn/ubuntu/ focal main restricted universe multiverse
3 deb https://mirrors.ustc.edu.cn/ubuntu/ focal-updates main restricted universe
  multiverse
4 deb-src https://mirrors.ustc.edu.cn/ubuntu/ focal-updates main restricted universe
  multiverse
5 deb https://mirrors.ustc.edu.cn/ubuntu/ focal-backports main restricted universe
  multiverse
6 deb-src https://mirrors.ustc.edu.cn/ubuntu/ focal-backports main restricted universe
  multiverse
7 deb https://mirrors.ustc.edu.cn/ubuntu/ focal-security main restricted universe
  multiverse
8 deb-src https://mirrors.ustc.edu.cn/ubuntu/ focal-security main restricted universe
  multiverse
9 deb https://mirrors.ustc.edu.cn/ubuntu/ focal-proposed main restricted universe
  multiverse
10 deb-src https://mirrors.ustc.edu.cn/ubuntu/ focal-proposed main restricted universe
    multiverse
```

按“ESC”按键退出编辑模式。然后按下“: wq!”进行保存退出。

更新系统:

```
1 sudo apt update
2 sudo apt upgrade
```

安装QEMU等必备包:

```
1 sudo apt-get install git build-essential gdb-multiarch qemu-system-misc gcc-riscv64-
  linux-gnu binutils-riscv64-linux-gnu
```

测试是否安装成功

```
1 riscv64-unknown-elf-gcc --version
2 qemu-system-riscv64 --version
```

如有报错，按照给的提示来解决。

2.安装XV6

将XV6代码克隆到本地。

```
1 git clone git://g.csail.mit.edu/xv6-labs-2020
```

进入克隆文件夹:

```
1 cd xv6-labs-2020/
```

切换到第一个实验分支

```
1 | git checkout util
```

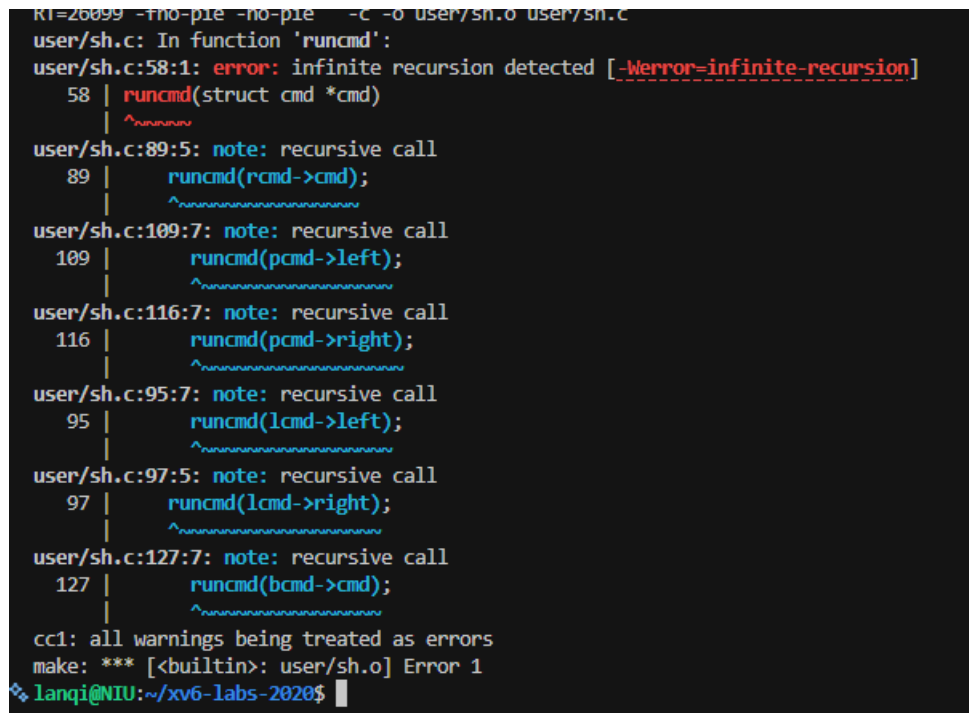
编译xv6并开启模拟器

```
1 | make qemu
```

按下 `ctrl+a` 松开后再按 `x` 退出qemu.

如果失败, 参考[Mit6.s081环境配置踩坑之旅WSL2+VScode mit6s081-CSDN博客](#)解决, 大概是因为Ubuntu系统的版本过高, 默认安装的qemu模拟器与xv6内核不兼容, 导致编译不成功。

此外, 实验过程中每个分支编译时都会出现:



```
RT=26099 -fno-pie -fno-pie -c -o user/sh.o user/sh.c
user/sh.c: In function 'runcmd':
user/sh.c:58:1: error: infinite recursion detected [-Werror=infinite-recursion]
  58 | runcmd(struct cmd *cmd)
      | ^~~~~~
user/sh.c:89:5: note: recursive call
  89 |     runcmd(rcmd->cmd);
      |     ^~~~~~
user/sh.c:109:7: note: recursive call
 109 |     runcmd(pcmd->left);
      |     ^~~~~~
user/sh.c:116:7: note: recursive call
 116 |     runcmd(pcmd->right);
      |     ^~~~~~
user/sh.c:95:7: note: recursive call
   95 |     runcmd(lcmd->left);
      |     ^~~~~~
user/sh.c:97:5: note: recursive call
   97 |     runcmd(lcmd->right);
      |     ^~~~~~
user/sh.c:127:7: note: recursive call
  127 |     runcmd(bcmd->cmd);
      |     ^~~~~~
cc1: all warnings being treated as errors
make: *** [<builtin>: user/sh.o] Error 1
lanqi@NIU:~/xv6-labs-2020$
```

这是新版 GCC 对 2020 年旧代码产生的兼容性警告。

解决:

在WSL的 xv6 项目目录中执行:

```
1 | cd ~/xv6-labs-2020
2 | nano Makefile
```

找到定义 `CFLAGS` 的位置:

```
1 | CFLAGS = -Wall -Werror -O -fno-omit-frame-pointer -ggdb
```

改为:

```
1 | CFLAGS = -Wall -Werror -Wno-error=infinite-recursion -O -fno-omit-frame-pointer -ggdb
```

再次编译即可。

三、关于 git 的使用

1. 建立自己的仓库

在 GitHub 上创建一个新仓库

<https://github.com/Lanqi2345/xv6-labs-2020.git>

2. 修改本地代码的“关联”

查看当前的关联（指向 MIT 的）：

```
1 | git remote -v
```

修改原来的关联的名字：

```
1 | git remote rename origin upstream
```

添加自己的仓库作为新的 `origin`：

```
1 | git remote add origin <仓库链接>
```

再次查看验证：

```
1 | git remote -v
```

`origin` 现在指向自己的账号

3. 推送

把本地的代码推送：

```
1 | # 假设当前在 util 分支
2 | git push -u origin util
```

Lab: Xv6 and Unix utilities

Boot xv6

要求：

本实验主要是搭建环境并熟悉 xv6 操作系统。

步骤：

实验环境搭建请参考“环境配置”部分，构建并运行 xv6：

切换到 `util` 分支

```
1 | git checkout util
2 | make qemu
```

```
xv6 kernel is booting

hart 2 starting
hart 1 starting
init: starting sh
$ |
```

sleep

要求:

为 xv6 操作系统实现一个用户态程序 `sleep`，使其能够暂停指定的时间片 (ticks) 数量。练习如何编写用户程序、处理命令行参数、使用系统调用以及将新程序集成到 xv6 构建系统中。

解决:

在 `user/` 目录下创建一个名为 `sleep.c` 的文件，写入：

```
#include "kernel/types.h"
#include "user/user.h"

int main(int argc, char *argv[])
{
    if(argc<2)
    {
        fprintf(2, "usage: sleep <ticks>\n"); //stderr 2, 标准错误, 报错、用消息提示专用
        exit(1); //0代表正常退出, 其他代表有错误
    }

    int ticks=atoi(argv[1]);

    if(sleep(ticks)<0) //返回 0: 表示操作成功。返回 -1 (或小于 0 的值): 表示操作失败
    {
        fprintf(2, "sleep: system call failed\n");
        exit(1);
    }
    exit(0);
}
```

在 `Makefile` 文件中，`UPROGS` 下添加 `$U/_sleep\`，并进行编译。

pingpong

要求:

为 xv6 操作系统实现一个用户态程序 `pingpong`，使用两条管道在父子进程之间双向传递一个字节。父进程发送字节后，子进程接收并打印 "`<pid>: received ping`"，再将该字节传回父进程；父进程接收后打印 "`<pid>: received pong`"。练习使用 `pipe`、`fork`、`read`、`write`、`getpid` 等 UNIX 系统调用，以及将新程序集成到 xv6 构建系统中。

解决:

在 `user/` 目录下创建一个名为 `pingpong.c` 的文件，实现：

1. 使用 `pipe()` 创建两条单向管道，分别用于父进程向子进程发送数据和子进程向父进程返回数据。
2. 使用 `fork()` 创建子进程。
3. 父子进程分别关闭自己不使用的管道端口。
4. 父进程通过第一条管道向子进程发送一个字节。
5. 子进程读取该字节，打印 `received ping`，再通过第二条管道将字节传回父进程并退出。
6. 父进程读取返回的字节，打印 `received pong`，使用 `wait()` 回收子进程后退出。
7. 对管道创建、进程创建和字节读写进行错误检查，并及时关闭使用完毕的文件描述符。

在 `Makefile` 文件的 `UPROGS` 下添加 `$U/_pingpong\`，并进行编译运行。

primes

要求：

为 xv6 实现用户态程序 `primes`，使用 `pipe` 和 `fork` 构建并发素数筛选管道。程序将 2~35 依次传入管道，每个进程负责筛选一个素数的倍数，最终输出所有素数。练习进程创建、管道通信和文件描述符管理。

解决：

在 `user/` 目录下创建一个名为 `primes.c` 的文件，实现：

1. 使用 `pipe()` 创建管道，并由主进程将 2~35 写入管道。
2. 使用 `fork()` 创建素数筛选进程。
3. 每个筛选进程读取第一个数字，将其作为素数输出。
4. 筛选掉该素数的倍数，并将剩余数字传给下一个进程。
5. 递归创建管道和子进程，直到没有数字需要处理。
6. 及时关闭不使用的管道端口，并使用 `wait()` 回收子进程。

在 `Makefile` 的 `UPROGS` 下添加 `$U/_primes\`，然后编译运行。

find

要求：

为 xv6 实现用户态程序 `find`，根据指定名称递归查找目录树中的文件，并输出匹配文件的完整路径。程序需要读取目录内容、区分文件和子目录、跳过 `.` 与 `..`，并使用 `strcmp()` 比较文件名。

解决：

在 `user/` 目录下创建一个名为 `find.c` 的文件，实现：

1. 使用 `open()` 打开指定路径，并通过 `fstat()` 获取文件类型。
2. 获取路径的最后一部分，并使用 `strcmp()` 与目标名称比较。
3. 如果名称匹配，则输出该文件或目录的完整路径。
4. 如果当前路径是目录，则使用 `read()` 依次读取其中的目录项。
5. 跳过无效目录项以及 `.` 和 `..`，避免无限递归。
6. 拼接子路径并递归调用 `find()`，继续查找子目录。

7. 检查路径长度，并及时关闭文件描述符。

在 `Makefile` 的 `UPROGS` 下添加 `$U/_find\`，然后编译运行。

xargs

要求：

为 xv6 实现用户态程序 `xargs`，逐行读取标准输入，将每行内容作为附加参数，与原有命令参数组合后执行该命令。程序需要使用 `fork()` 创建子进程、使用 `exec()` 执行命令，并由父进程通过 `wait()` 等待命令完成，进行标准输入处理、命令参数构造和进程控制。

解决：

在 `user/` 目录下创建一个名为 `xargs.c` 的文件，实现：

1. 保存 `xargs` 后原有的命令及参数。
2. 使用 `read()` 从标准输入逐字符读取内容，并按换行符划分输入行。
3. 将每行内容按空格或制表符拆分成附加参数。
4. 将原有参数和附加参数组合成新的参数数组，并在末尾添加空指针。
5. 使用 `fork()` 创建子进程，并通过 `exec()` 执行组合后的命令。
6. 父进程使用 `wait()` 等待子进程结束，再处理下一行输入。
7. 检查参数数量、输入长度以及进程创建和命令执行错误。

在 `Makefile` 的 `UPROGS` 下添加 `$U/_xargs\`，然后编译运行。

Lab总结

实验结果：

```
== Test sleep, no arguments ==
$ make qemu-gdb
sleep, no arguments: OK (2.4s)
== Test sleep, returns ==
$ make qemu-gdb
sleep, returns: OK (0.7s)
== Test sleep, makes syscall ==
$ make qemu-gdb
sleep, makes syscall: OK (1.0s)
== Test pingpong ==
$ make qemu-gdb
pingpong: OK (1.0s)
== Test primes ==
$ make qemu-gdb
primes: OK (1.0s)
== Test find, in current directory ==
$ make qemu-gdb
❖ find, in current directory: OK (0.9s)
== Test find, recursive ==
$ make qemu-gdb
find, recursive: OK (1.1s)
== Test xargs ==
$ make qemu-gdb
xargs: OK (1.0s)
== Test time ==
time: OK
Score: 100/100
lanqi@NIU:~/xv6-labs-2020$
```

问题与解决

在本阶段的实验中，最大的问题就是我一开始无法理解函数之间的调用逻辑。比如在编写 `sleep.c` 时，我无法区分自己编写的用户程序与系统原本提供的 `sleep()` 函数，也不理解用户程序如何调用内核函数，以及在命令行中输入命令后，系统如何运行 `sleep.c` 中的 `main()` 函数。通过阅读书籍以及查阅相关资料，我对此有了初步认识：

当在 xv6 的 shell 中输入 `sleep 10` 时，shell 会解析命令和参数，然后通过 `fork()` 创建子进程，并使用 `exec()` 加载已经编译好的 `sleep` 用户程序。程序开始运行后，首先进入 `main()` 函数，其中 `argc` 表示参数数量，`argv` 用于保存命令及其参数。程序通过 `atoi()` 将字符串形式的 "10" 转换为整数，再调用系统提供的 `sleep()`。

我编写的 `sleep.c` 是一个完整的用户态命令，而其中调用的 `sleep()` 是 xv6 提供的系统调用，两者虽然名称相同，但作用不同。用户程序中的 `sleep()` 调用会通过系统调用入口进入内核，由内核中的 `sys_sleep()` 完成真正的进程暂停操作。用户程序不能直接执行内核函数，而是需要通过系统调用完成从用户态到内核态的切换。理解这一过程后，我对用户程序、系统调用和内核函数之间的关系有了更加清晰的认识。

在 `user/sleep.c` 中调用 `sleep(ticks)` 时，它的实现过程可以分为以下三个层次。

第一层：函数声明

在 `sleep.c` 中包含：

```
1 | #include "user/user.h"
```

编译器会读取 `user/user.h` 中的声明：

```
1 | int sleep(int);
```

这条声明告诉编译器，系统中存在一个名为 `sleep` 的函数，它接收一个整数参数并返回一个整数。头文件只说明函数应该如何调用，并不包含真正的实现。

第二层：用户态系统调用存根

系统调用的用户态入口位于 `user/usys.s` 中。这里使用一小段汇编代码完成系统调用号的设置和用户态到内核态的切换，其逻辑类似于：

```
1 | .global sleep
2 | sleep:
3 |     li a7, SYS_sleep
4 |     ecall
5 |     ret
```

其中，`li a7, SYS_sleep` 将 `sleep` 对应的系统调用号放入 `a7` 寄存器；`ecall` 指令触发陷阱，使 CPU 从用户模式进入内核模式；内核处理完成后，`ret` 再返回用户程序。

第三层：内核中的实际处理函数

进入内核后，系统会读取 `a7` 寄存器中的系统调用号，并根据系统调用表找到对应的处理函数。`sleep` 最终会调用位于 `kernel/sysproc.c` 中的 `sys_sleep`，这里是真正完成进程休眠操作的地方。用户程序不能直接调用内核函数，而是必须通过系统调用接口，请求内核提供相应服务。

实验心得

通过本次实验，我对 xv6 用户程序的编写、编译和运行过程有了更加清晰的认识。sleep 实验让我掌握了命令行参数处理和系统调用的基本使用方法；pingpong 实验帮助我理解了管道的单向通信特点以及父子进程之间的数据交换；primes 实验进一步展示了如何利用进程和管道组成并发流水线。在 find 实验中，我学习了目录项读取、文件类型判断、路径拼接和递归遍历等文件系统操作；在 xargs 实验中，我掌握了标准输入读取、字符串拆分、参数数组构造以及 fork()、exec() 和 wait() 的配合使用。

这次实验，让我对操作系统有了进一步的认知与理解，也增加了对某些命令行的理解，比如，之前知道管道运算符与 xargs 命令，但从未了解命令本身的实现原理。

Lab: system calls

System call tracing

要求：

为 xv6 内核添加系统调用跟踪功能，实现新的 trace 系统调用。程序通过掩码指定需要跟踪的系统调用，内核在对应系统调用返回时输出进程 PID、系统调用名称和返回值。跟踪设置需要对调用进程及其后续创建的子进程生效，但不能影响其他进程。实现添加系统调用、修改进程结构、继承进程状态以及理解系统调用的执行流程。

解决：

1. 切换到 syscall 分支

```
1 | git checkout syscall
```

2. 把 trace 用户程序加入编译

在 Makefile 的 UPROGS 下添加 \$U/_trace\。

3. 在 user/user.h 声明 trace()

因为 user/trace.c 需要调用 trace(mask)；

```
1 | int trace(int);
```

4. 在 user/usys.pl 添加系统调用桩

添加

```
1 | entry("trace");
```

编译时，usys.pl 会自动生成 user/usys.S，负责把 trace 的系统调用编号放入 a7；使用 ecall 从用户态进入内核；系统调用结束后返回用户态。

5. 给 trace 分配系统调用编号

在 kernel/syscall.h 添加：

```
1 | #define SYS_trace 22
```

6. 在进程结构中添加 trace_mask

在 kernel/proc.h 中的 struct proc 添加

```
1 | int trace_mask;
```

让每个进程都有自己独立的跟踪掩码。

7. 初始化 trace_mask

在 kernel/proc.c 中的 allocproc 添加

```
1 | p->trace_mask = 0;
```

8. 在内核中实现 sys_trace()

在 kernel/sysproc.c 中添加:

```
uint64 sys_trace(void)
{
    int mask;
    if(argint(0, &mask) < 0) //从用户参数中取出掩码
        return -1;
    myproc()->trace_mask = mask; //将掩码保存进当前进程
    return 0;
}
```

9. 向内核声明 sys_trace()

在 kernel/syscall.c 中添加

```
1 | extern uint64 sys_trace(void);
```

10. 把编号22映射到 sys_trace()

在 kernel/syscall.c 中添加

```
1 | [SYS_trace] sys_trace,
```

11. 让 fork() 复制 trace_mask

在 kernel/proc.c 的 fork 中添加

```
1 | // 子进程继承父进程的跟踪掩码
2 | np->trace_mask = p->trace_mask;
```

12. 添加系统调用名称数组

在 kernel/syscall.c 中添加:

```

136 static char *syscall_names[] = {
137     [SYS_fork] = "fork",
138     [SYS_exit] = "exit",
139     [SYS_wait] = "wait",
140     [SYS_pipe] = "pipe",
141     [SYS_read] = "read",
142     [SYS_kill] = "kill",
143     [SYS_exec] = "exec",
144     [SYS_fstat] = "fstat",
145     [SYS_chdir] = "chdir",
146     [SYS_dup] = "dup",
147     [SYS_getpid] = "getpid",
148     [SYS_sbrk] = "sbrk",
149     [SYS_sleep] = "sleep",
150     [SYS_uptime] = "uptime",
151     [SYS_open] = "open",
152     [SYS_write] = "write",
153     [SYS_mknod] = "mknod",
154     [SYS_unlink] = "unlink",
155     [SYS_link] = "link",
156     [SYS_mkdir] = "mkdir",
157     [SYS_close] = "close",
158     [SYS_trace] = "trace",
159     [SYS_sysinfo] = "sysinfo",
160 };

```

13. 在 `syscall()` 中检查掩码

在 `kernel/syscall.c` 中添加

```

void
syscall(void)
{
    int num;
    struct proc *p = myproc();

    num = p->trapframe->a7;
    if(num > 0 && num < NELEM(syscalls) && syscalls[num]) {
        p->trapframe->a0 = syscalls[num]();

        if (p->trace_mask & (1U << num))
        {
            printf("%d: syscall %s -> %d\n",
                p->pid,
                syscall_names[num],
                (int)p->trapframe->a0);
        }
    } else {
        printf("%d %s: unknown sys call %d\n",
            p->pid, p->name, num);
        p->trapframe->a0 = -1;
    }
}

```

Sysinfo

要求:

为 xv6 内核添加 `sysinfo` 系统调用，用于获取系统当前运行状态。内核需要统计空闲内存的字节数和状态不为 `UNUSED` 的进程数量，将结果写入 `struct sysinfo` 并复制到用户空间。实现添加系统调用、统计内核资源以及使用 `copyout()` 在内核与用户空间之间传递数据。

解决:

1. 把测试程序加入编译

在 `Makefile` 的 `UPROGS` 下添加:

```
1 | $U/_sysinfotest\
```

2. 在 `user/user.h` 声明 `sysinfo()`

```
1 | struct sysinfo;  
2 | int sysinfo(struct sysinfo *);
```

`struct sysinfo;` 是结构体的前向声明。

3. 在 `user/usys.pl` 添加系统调用桩

```
1 | entry("sysinfo");
```

编译时会自动生成汇编桩, 通过 `ecall` 从用户态进入内核。

4. 给 `sysinfo` 分配系统调用编号

在 `kernel/syscall.h` 中添加:

```
1 | #define SYS_sysinfo 23
```

5. 在内核中声明 `sys_sysinfo()`

在 `kernel/syscall.c` 中添加:

```
1 | extern uint64 sys_sysinfo(void);
```

6. 注册 `sysinfo` 系统调用

在 `kernel/syscall.c` 的 `syscalls[]` 数组中添加:

```
1 | [SYS_sysinfo] sys_sysinfo,
```

对于已经完成的 `trace` 实验, 还需要在系统调用名称数组中添加:

```
1 | [SYS_sysinfo] = "sysinfo",
```

7. 声明资源统计函数

在 `kernel/defs.h` 中添加:

```
1 | uint64 freemem(void);  
2 | int nproc(void);
```

分别用于统计空闲内存和系统进程数量。

8. 实现统计空闲内存

在 `kernel/kalloc.c` 中遍历空闲页链表，每个空闲链表节点代表一个物理页，因此每找到一个节点就增加 `PGSIZE` 字节。

```
uint64 freemem(void)
{
    struct run *r;
    uint64 bytes=0;

    acquire(&kmem.lock);
    r=kmem.freelist;
    while(r!=0){
        bytes+=PGSIZE;
        r=r->next;
    }
    release(&kmem.lock);
    return bytes;
}
```

9. 实现统计进程数量

在 `kernel/proc.c` 中遍历进程表，统计所有状态不为 `UNUSED` 的进程。

```
int nproc(void)
{
    struct proc *p;
    int count = 0;

    for(p = proc; p < &proc[NPROC]; p++){
        acquire(&p->lock);
        if(p->state != UNUSED) {
            count++;
        }
        release(&p->lock);
    }

    return count;
}
```

10. 实现 `sys_sysinfo()`

在 `kernel/sysproc.c` 中包含：

```
1 | #include "sysinfo.h"
```

然后实现系统调用：

- `argaddr()` 获取用户传入的结构体地址；
- `freemem()` 获取空闲内存字节数；
- `nproc()` 获取当前进程数量；

- `copyout()` 将内核中的统计结果复制到用户空间。

```
uint64 sys_sysinfo(void)
{
    struct sysinfo info;
    uint64 user_address;
    struct proc *p = myproc();

    if(argaddr(0,&user_address)<0)
        return -1;

    info.freemem = freemem();
    info.nproc = nproc();

    if(copyout(p->pagetable, user_address, (char*)&info, sizeof(info)) < 0)
        return -1;

    return 0;
}
```

Lab总结

实验结果：

```
== Test trace 32 grep ==
$ make qemu-gdb
trace 32 grep: OK (2.5s)
== Test trace all grep ==
$ make qemu-gdb
trace all grep: OK (0.6s)
== Test trace nothing ==
$ make qemu-gdb
trace nothing: OK (1.0s)
== Test trace children ==
$ make qemu-gdb
trace children: OK (7.1s)
== Test sysinfotest ==
$ make qemu-gdb
sysinfotest: OK (0.8s)
== Test time ==
time: OK
Score: 35/35
lanqi@NIU:~/xv6-labs-2020$
```

问题与解决

1. 新增系统调用后无法编译

在加入 `trace` 和 `sysinfo` 后，用户程序出现函数未声明或找不到系统调用入口的问题。原因是新增系统调用需要同时连接用户态和内核态。

解决方法是在以下文件中补充相应内容：

- 在 `user/user.h` 中声明系统调用；
- 在 `user/usys.pl` 中添加系统调用桩；
- 在 `kernel/syscall.h` 中分配不重复的系统调用编号；

- 在 `kernel/syscall.c` 中声明内核函数并加入系统调用映射表。

完成这些步骤后，用户程序才能通过 `ecall` 正确进入内核。

2. `sysinfo()` 无法将结果返回用户空间

`sys_sysinfo()` 中统计出的数据位于内核空间，不能直接通过用户传入的指针进行赋值。解决方法是先使用 `argaddr()` 获取用户空间地址，再使用 `copyout()` 根据当前进程的页表将 `struct sysinfo` 复制到用户空间。

实验心得

通过本次实验，我进一步理解了 xv6 系统调用从用户态进入内核态的完整过程。一个新的系统调用不仅需要实现内核函数，还需要完成用户态声明、汇编调用桩、系统调用编号和内核分发表等多个环节。任何一个环节缺失，都可能造成编译、链接或运行错误。

总体来说，这两个实验让我对 xv6 的系统调用机制、进程管理、内存管理以及内核并发控制有了更深入的认识，也提高了我根据编译错误和测试结果逐步定位问题的能力。

Lab: page tables

Print a page table

要求：

为 xv6 内核实现 `vmprint()` 函数，递归遍历并打印指定的三级 RISC-V 页表。输出需要包含有效页表项的层级、索引、PTE 值和对应物理地址，并在第一个进程完成 `exec()` 时打印其页表。从而实现理解多级页表结构、页表项标志、虚拟地址映射以及用户访问权限。

解决：

1. 切换到 `pgtbl` 分支

```
1 | git checkout pgtbl
```

2. 参考页表释放函数

在 `kernel/vm.c` 中查看 `freewalk()`。

了解到判断 PTE 有效：

```
1 | pte & PTE_V
```

判断 PTE 指向下一级页表：

```
1 | (pte & (PTE_R | PTE_W | PTE_X)) == 0
```

3. 实现递归遍历函数与 `vmprint()` 函数

在 `kernel/vm.c` 中添加：

```

//递归打印
static void
vmprintwalk(pagetable_t pagetable, int level)
{
    for(int i = 0; i < 512; i++)//遍历当前页表的512个 PTE
    {
        pte_t pte = pagetable[i];

        if(pte & PTE_V) //只处理有效PTE
        {
            uint64 pa = PTE2PA(pte);//使用PTE2PA()提取物理地址

            //根据页表深度打印缩进
            for (int j = 1; j < level; j++)
                printf(".. ");

            printf("..%d: pte %p pa %p\n",i, pte, pa);

            //如果R/W/X都没有设置，说明该PTE指向下一级页表，而不是最终物理页面。
            if ((pte & (PTE_R | PTE_W | PTE_X)) == 0){
                vmprintwalk((pagetable_t)pa, level + 1);//递归访问
            }
        }
    }
}

void
vmprint(pagetable_t pagetable)
{
    printf("page table %p\n", pagetable);
    vmprintwalk(pagetable, 1);
}

```

4. 声明 `vmprint()`

在 `kernel/defs.h` 的 `vm.c` 函数声明区域添加:

```
1 | void vmprint(pagetable_t);
```

5. 在第一个进程执行 `exec()` 后打印页表

添加:

```

if(p->pid==1) vmprint(p->pagetable);
return argc; // this ends up in a0, the first argument to main(argc, argv)

```

A kernel page table per process

要求:

修改 xv6 内核，为每个进程创建并维护一份独立的内核页表，使进程进入内核态后使用自己的内核页表。需要为页表建立必要的内核映射和进程内核栈映射，在进程调度时切换页表，并在没有进程运行时恢复全局内核页表，最后在进程释放时正确释放其内核页表。实现页表创建、内核栈映射、进程调度和内存资源管理。

解决：

1. 在进程结构中保存独立内核页表

在 `kernel/proc.h` 的 `struct proc` 中添加：

```
1 | pagetable_t kpagetable; // 进程独立的内核页表
```

2. 在 `kernel/defs.h` 声明相关函数

将 `kvmmap()` 修改为可以操作指定页表，并添加创建、释放函数：

```
1 | pagetable_t kvmmake(void); //创建并返回内核页表
2 | void kvmmap(pagetable_t, uint64, uint64, uint64, int); //接收页表参数
3 | void kvmfree(pagetable_t); //进程退出后需要释放内核页表
```

3. 修改 `kvmmap()`

在 `kernel/vm.c` 中，将原来只能操作全局内核页表的函数改为：

```
1 | void kvmmap(pagetable_t pagetable, uint64 va, uint64 pa, uint64 sz, int perm)
2 | {
3 |     if (mappages(pagetable, va, sz, pa, perm) != 0)
4 |         panic("kvmmap");
5 | }
```

这样既可以修改全局内核页表，也可以修改某个进程的内核页表。

4. 实现 `kvmmake()`

在 `kernel/vm.c` 中实现创建内核页表的函数，映射内容参考 `kvminit()` 中的映射。

```

//创建并返回内核页表
pagetable_t kvmmake(void)
{
    pagetable_t pagetable;

    pagetable = (pagetable_t)kalloc();
    if (pagetable == 0)
        return 0;

    memset(pagetable, 0, PGSIZE);

    // uart registers
    kvmmap(pagetable, UART0, UART0, PGSIZE, PTE_R | PTE_W);

    // virtio mmio disk interface
    kvmmap(pagetable, VIRTIO0, VIRTIO0, PGSIZE, PTE_R | PTE_W);

    // CLINT
    kvmmap(pagetable, CLINT, CLINT, 0x10000, PTE_R | PTE_W);

    // PLIC
    kvmmap(pagetable, PLIC, PLIC, 0x400000, PTE_R | PTE_W);

    // map kernel text executable and read-only.
    kvmmap(pagetable, KERNBASE, KERNBASE, (uint64)etext-KERNBASE, PTE_R | PTE_X);

    // map kernel data and the physical RAM we'll make use of.
    kvmmap(pagetable, (uint64)etext, (uint64)etext, PHYSTOP-(uint64)etext, PTE_R | PTE_W);

    // map the trampoline for trap entry/exit to
    // the highest virtual address in the kernel.
    kvmmap(pagetable, TRAMPOLINE, (uint64)trampoline, PGSIZE, PTE_R | PTE_X);

    return pagetable;
}

```

5. 修改 `kvminit()`

在 `kernel/vm.c` 中使用 `kvmmake()` 创建全局内核页表:

```

1 void
2 kvminit(void)
3 {
4     kernel_pagetable = kvmmake();
5     if (kernel_pagetable == 0)
6         panic("kvminit");
7 }

```

6. 修改 `procinit()`

在 `kernel/proc.c` 的 `procinit()` 中, 删除提前为所有进程分配和映射内核栈的代码, 只初始化进程锁和相关字段, 内核栈改为在真正创建进程时分配。

```

1  for (p = proc; p < &proc[NPROC]; p++) {
2      initlock(&p->lock, "proc");
3      p->kstack = 0;
4      p->kpagetable = 0;
5  }

```

7. 在 `allocproc()` 中创建进程内核页表，为进程分配并映射内核栈

在 `kernel/proc.c` 的 `allocproc()` 中加入：

```

1  // 为新进程创建独立的内核页表，并将内核所需要的各种映射添加到新页表上
2  p->kpagetable = kvmmake();

```

在 `allocproc()` 中分配内核栈物理页：

```

1  // 分配一个物理页，作为新进程的内核栈使用
2  char *pa = kalloc();
3  if(pa == 0)
4      panic("kalloc");
5  uint64 va = KSTACK((int) (p - proc)); // 将内核栈映射到固定的逻辑地址上
6  kvmmap(p->kpagetable, va, (uint64)pa, PGSIZE, PTE_R | PTE_W);
7  p->kstack = va;

```

8. 实现内核页表释放函数

在 `kernel/vm.c` 中添加：

```

//进程退出后释放内核页表
void kvmfree(pagetable_t pagetable)
{
    for(int i=0;i<512;i++)
    {
        pte_t pte = pagetable[i];
        if((pte & PTE_V) == 0)
            continue;

        //不是叶子，指向下一级页表
        if((pte & (PTE_R | PTE_W | PTE_X)) == 0)
        {
            uint64 child = PTE2PA(pte);
            kvmfree((pagetable_t)child);
        }

        //清除页表项，不释放物理页
        pagetable[i] = 0;
    }

    kfree((void*)pagetable);
}

```

该函数只释放页表结构页，不释放内核代码、设备地址等叶子 PTE 指向的共享物理内存。

9. 在 `freeproc()` 中释放内核栈和页表

在 `kernel/proc.c` 的 `freeproc()` 中添加:

```
1 //先释放内核栈物理页
2 if (p->kpagetable && p->kstack)
3 {
4     uvmunmap(p->kpagetable, p->kstack, 1, 1);
5     p->kstack = 0;
6 }
7
8 //释放页表结构页，不释放共享内核叶子物理页。
9 if (p->kpagetable)
10 {
11     kvmfree(p->kpagetable);
12     p->kpagetable = 0;
13 }
```

10. 在调度器中切换内核页表

在 `kernel/proc.c` 的 `scheduler()` 中, 修改:

```
1 //使用p自己的内核页表
2 w_satp(MAKE_SATP(p->kpagetable));
3 sfence_vma(); //清除旧页表对应的 TLB 缓存
4
5 swtch(&c->context, &p->context);
6
7 //恢复全局内核页表
8 kvmminithart();
```

`sfence_vma()` 用于清除旧页表对应的 TLB 缓存。

11. 修改 `kvmpa()`

在 `kernel/vm.c` 中, 让 `kvmpa()` 优先使用当前进程的内核页表:

```
1 struct proc *p = myproc();
2 pagetable_t pagetable;
3
4 if (p != 0 && p->kpagetable != 0)
5     pagetable = p->kpagetable;
6 else
7     pagetable = kernel_pagetable;
8
9 pte = walk(pagetable, va, 0);
```

12. 修改 `kernel/vm.c` 的头文件

在 `kernel/vm.c` 中加入:

```
1 | #include "spinlock.h"
2 | #include "proc.h"
```

Simplify

要求:

在上一题的基础上，将每个进程的用户内存映射同步到其独立内核页表，使内核能够直接访问用户虚拟地址，并使用 `copyin_new()` 和 `copyinstr_new()` 简化用户数据复制。需要在 `userinit()`、`fork()`、`exec()` 和 `sbrk()` 等用户地址空间发生变化的位置同步页表映射，清除内核页表中用户页面的 `PTE_U` 权限，并限制进程地址空间不能超过 `PLIC` 地址，同时理解地址加法溢出检查的作用。

解决:

1. 实现用户映射同步函数

在 `kernel/vm.c` 中添加:

```
//实现用户映射同步函数
int u2kvmcopy(pagetable_t user_pagetable, pagetable_t kernel_pagetable, uint64 oldsz, uint64 newsz){
    //printf("u2kvmcopy: pid=%d old=%p new=%p\n", myproc()->pid, oldsz, newsz);
    uint64 start;
    uint64 va;
    pte_t *pte;
    uint64 pa;
    uint flags;

    if (newsz < oldsz)
        return -1;
    // 用户地址不能达到PLIC, 否则会与内核设备映射重叠
    if (PGROUNDUP(newsz) >= PLIC)
        return -1;
    // PGROUNDUP(x)会将地址向上取整到最近的页边界
    start = PGROUNDUP(oldsz);

    for (va = start; va < newsz; va += PGSIZE){
        pte = walk(user_pagetable, va, 0);

        if (pte == 0)
            panic("u2kvmcopy: pte");
        if ((*pte & PTE_V) == 0)
            panic("u2kvmcopy: not present");

        pa = PTE2PA(*pte);

        //使用用户PTE原有权限, 但清除PTE_U, 在Supervisor模式不能直接访问PTE_U=1的页面。
        flags = PTE_FLAGS(*pte);
        flags &= ~PTE_U;

        if (mappages(kernel_pagetable, va, PGSIZE, pa, flags) != 0){
            if (va > start) //映射一半, 失败时的回滚, 解除映射
                uvmunmap(kernel_pagetable, start, (va - start) / PGSIZE, 0);
            return -1;
        }
    }
    return 0;
}
```

这个函数让两个页表中的叶子 PTE 指向同一个物理页，而且必须清除内核页表中的 `PTE_U`，否则 RISC-V 内核态不能直接访问该映射。

2. 在 kernel/defs.h 声明函数 u2kvmcopy

```
1 | int u2kvmcopy(pagetable_t, pagetable_t, uint64, uint64);
```

3. 为第一个进程同步用户映射

在 kernel/proc.c 的 userinit() 中添加:

```
1 | //用户地址的内核映射
2 | if (u2kvmcopy(p->pagetable, p->kpagetable, 0, p->sz) < 0) {
3 |     panic("userinit: u2kvmcopy");
4 | }
```

给第一个 init 进程的内核页表中添加用户内存映射。

4. 在 fork() 中同步子进程映射

在 kernel/proc.c 的 fork() 中, 完成用户页表复制后添加:

```
1 | //把子进程用户页表中的用户页面, 映射到子进程自己的内核页表中。
2 | if (u2kvmcopy(np->pagetable, np->kpagetable, 0, np->sz) < 0){
3 |     freeproc(np);
4 |     release(&np->lock);
5 |     return -1;
6 | }
```

5. 限制用户地址空间大小

在 kernel/vm.c 的 uvmalloc() 中开始部分添加:

```
1 | if (newsz >= PLIC)
2 |     return 0;
```

因为用户虚拟地址不能达到 PLIC, 否则会与内核页表中的设备映射发生冲突。

6. 修改 growproc() 处理内存增长

在 kernel/proc.c 的 growproc() 中, 当 sbrk() 扩大地址空间时, 同步新增映射, 扩大内存时建立新的内核别名映射; 缩小内存时删除对应映射, 但不能再次释放物理页, uvmunmap() 的最后一个参数是 0。


```

// Grow or shrink user memory by n bytes.
// Return 0 on success, -1 on failure.
int
growproc(int n)
{
    uint64 sz;
    uint64 oldsz;
    struct proc *p = myproc();

    oldsz = p->sz;
    sz = oldsz;

    if (n > 0){
        //防止用户地址增长到PLIC
        if ((uint64)n > PLIC - oldsz)
            return -1;
        if ((sz = uvmalloc(p->pagetable, oldsz, oldsz + n)) == 0)
            return -1;
        if (u2kvmcopy(p->pagetable, p->kpagetable, oldsz, sz) < 0){
            uvmdealloc(p->pagetable, sz, oldsz);
            return -1;
        }
    }
    else if (n < 0) {
        //释放用户物理页
        sz = uvmdealloc(p->pagetable, oldsz, oldsz + n);

        uint64 oldend = PGROUNDUP(oldsz);
        uint64 newend = PGROUNDUP(sz);

        if (newend < oldend){
            //删除内核页表中的别名映射，
            uvmunmap(p->kpagetable, newend, (oldend - newend) / PGSIZE, 0);
        }
    }
    p->sz = sz;
    return 0;
}

```

7. 在 `exec()` 中同步新程序映射

在 `kernel/exec.c` 中，所有 ELF 加载、用户栈构造和参数复制成功后，先删除旧程序在内核页表中的映射，然后添加新程序的映射：

```

1  oldsz = p->sz;
2  //删除旧用户地址在内核页表中的映射
3  if (oldsz > 0){
4      uvmunmap(p->kpagetable, 0, PGROUNDUP(oldsz) / PGSIZE, 0);
5  }
6  //新用户页表映射同步到内核页表
7  if (u2kvmcopy(pagetable, p->kpagetable, 0, sz) < 0){
8      //恢复旧映射，保证exec失败时旧程序仍能运行。

```

```

9 |     if (oldsz > 0 && u2kvmcopy(p->pagetable, p->kpagetable, 0, oldsz) < 0){
10 |         //恢复失败
11 |         panic("exec: restore kpagetable");
12 |     }
13 |     goto bad; //失败继续执行旧程序
14 | }

```

8. 使用 `copyin_new()`

在 `kernel/vm.c` 中, 将 `copyin()` 的函数体替换为:

```

1 | //把数据复制进内核
2 | int copyin(pagetable_t pagetable, char *dst, uint64 srcva, uint64 len)
3 | {
4 |     return copyin_new(pagetable, dst, srcva, len);
5 | }

```

9. 使用 `copyinstr_new()`

将 `copyinstr()` 的函数体替换为:

```

1 | int copyinstr(pagetable_t pagetable, char *dst, uint64 srcva, uint64 max){
2 |     return copyinstr_new(pagetable, dst, srcva, max);
3 | }

```

10. 从每进程内核页表中删除 `CLINT` 映射

```

1 | // CLINT
2 | //CLINT 映射冲突
3 | //kvmmap(pagetable, CLINT, CLINT, 0x10000, PTE_R | PTE_W);

```

同时在 `kvminit()` 中添加:

```

1 | //单独给全局内核页表映射 CLINT
2 | kvmmap(kernel_pagetable, CLINT, CLINT, 0x10000, PTE_R | PTE_W);

```

对于题目中的两个问题:

创建 `answers-pgtbl.txt` 并回答:

问题一:

`vmprint` 输出的是三级页表结构。

第 0 页存放 `init` 进程的程序代码和数据。

第 1 页是用户栈的保护页。该页面对应的页表项没有设置 `PTE_U` 标志, 因此进程在用户模式下不能读取或写入该页面。如果用户程序尝试访问它, CPU 将产生页面错误。

第 2 页存放用户栈。

问题二:

三个判断为 `srcva >= p->sz || srcva+len >= p->sz || srcva+len < srcva`

`srcva + len < srcva` 用于检测无符号 64 位整数加法是否发生溢出。

例如，假设进程大小 `p->sz` 为 4096，并令：

```
srcva = 1
len = 0xffffffffffffffff
```

在数学上，两者之和超出了 `uint64` 能表示的范围。由于无符号整数运算会回绕，`srcva + len` 的实际结果会变成 0。

此时 `srcva` 小于 `p->sz`，并且溢出后的 `srcva + len` 也小于 `p->sz`，因此前面的边界检查可能无法发现问题。

但是 `srcva + len < srcva` 等价于：`0 < 1`，结果为真，因此可以检测出此次整数溢出并返回错误，防止内核访问错误的用户地址。

Lab总结

实验结果：

```
== Test pte printout ==
$ make qemu-gdb
pte printout: OK (2.1s)
== Test answers-pttbl.txt == answers-pttbl.txt: OK
== Test count copyin ==
$ make qemu-gdb
count copyin: OK (0.8s)
== Test usertests ==
$ make qemu-gdb
(61.6s)
== Test usertests: copyin ==
usertests: copyin: OK
== Test usertests: copyinstr1 ==
usertests: copyinstr1: OK
== Test usertests: copyinstr2 ==
usertests: copyinstr2: OK
== Test usertests: copyinstr3 ==
usertests: copyinstr3: OK
== Test usertests: sbrkmuch ==
usertests: sbrkmuch: OK
== Test usertests: all tests ==
usertests: all tests: OK
== Test time ==
time: OK
Score: 66/66
lanqi@NIU:~/xv6-labs-2020$
```

问题与解决

1. 页表打印格式和递归层级不正确

在实现 `vmprint()` 时，最初不容易区分叶子页表项与指向下一级页表的页表项。如果对所有有效 PTE 都递归，会把普通物理页误当作页表访问，导致异常。

解决方法是参考 `freewalk()`：首先通过 `PTE_V` 判断页表项是否有效，再检查 `PTE_R`、`PTE_W` 和 `PTE_X`。当三个位都没有设置时，说明该 PTE 指向下一级页表，可以递归遍历。同时通过递归深度控制 `..` 的数量，使输出格式符合测试要求。

2. CLINT 映射与用户地址空间发生冲突

创建进程独立内核页表时，最初按照全局内核页表复制了 CLINT 映射：

```
1 //CLINT
2 kvmmap(pagetable, CLINT, CLINT, 0x10000, PTE_R | PTE_W);
```

CLINT 位于较低的虚拟地址。但是在第三个任务中，将用户地址映射复制到进程内核页表，用户地址空间增长后可能与 CLINT 映射重叠，导致 `mappages()` 报错或产生映射冲突。于是在进程的独立内核页表中取消 CLINT 映射：

```
1 //CLINT 映射冲突
2 //kvmmap(pagetable, CLINT, CLINT, 0x10000, PTE_R | PTE_W);
```

CLINT 主要由机器模式下的定时器初始化代码访问，不需要出现在进程运行时使用的内核页表中。移除该映射后，进程内核页表中最低的内核映射地址变为 `PLIC`，因此用户地址空间必须限制在 `PLIC` 以下。

但是还要保留全局页表需要的映射：

```
1 //单独给全局内核页表映射 CLINT
2 kvmmap(kernel_pagetable, CLINT, CLINT, 0x10000, PTE_R | PTE_W);
```

实验心得

这次实验是我消耗时长最长的一个实验，最初进行实验时，由于对用户态以及内核态本身的理解不够，我不太理解为什么需要为每个进程重新创建一张内核页表。原来的 xv6 已经具有全局内核页表，而每个进程也有自己的用户页表，看起来已经能够正常完成地址转换。

结合题目要求和代码分析后，我理解到：原来的全局内核页表没有当前进程的用户地址映射，因此内核不能直接解引用系统调用传入的用户指针。`copyin()` 必须先通过用户页表查询物理地址，再访问对应的物理内存。

本实验要求先为每个进程创建独立的内核页表，并在调度进程时切换到该页表；然后将该进程的用户内存映射同步到它的内核页表。这样，内核态和用户态可以使用相同的用户虚拟地址访问同一个物理页，`copyin_new()` 和 `copyinstr_new()` 就能够直接访问用户指针，不再需要在软件中逐级遍历用户页表。

通过实验，我逐渐明确了三个任务之间的关系：

1. `vmprint()` 用于观察和理解三级页表结构。
2. 每进程内核页表为不同进程建立独立的内核运行地址空间。
3. 将用户映射加入进程内核页表，使内核可以直接访问当前进程的用户内存。

本次实验涉及页表创建、进程调度、内存增长、程序加载和进程释放等多个模块。它让我体会到内核中的修改往往会影响整个系统，编写代码时必须重视资源生命周期、失败回滚、权限控制和边界检查等内容。

Lab: traps

RISC-V assembly

要求：

通过阅读 `user/call.asm` 分析 RISC-V 函数调用和汇编代码，理解函数参数寄存器、`ra` 返回地址、编译器内联、函数地址以及大小端字节序。同时分析 `printf()` 参数数量不匹配时的输出现象，并将相关问题的答案写入 `answers-traps.txt`。

解决:

这个小问主要是读汇编并回答问题, 所以:

1. 切换到 `traps` 分支

```
1 | git checkout traps
```

2. 在仓库根目录创建 `answers-traps.txt`, 并回答问题:

```
1 | 1.RISC-V 函数参数存放在寄存器a0到a7中。a2中。
2 |
3 | 2.main的汇编代码中通常没有真正调用f和g。编译器将f和g内联并进一步进行了常量计算, 因此直接得到结果。
4 |
5 | 3.printf位于地址0x616。
6 |
7 | 4.jalr调用函数时, 会把“下一条指令的地址”保存到ra, 以便printf执行完以后返回。
8 | ra=0x38
9 |
10 | 5.程序输出: He110 world, 57616的十六进制表示是0xe110, 因此%x输出e110。
11 | 在小端RISC-V中, i = 0x00646c72在内存中的字节排列为:
12 | 72 6c 64 00
13 | 对应字符串"rld\0", 所以"wo%s"输出"world"。
14 | 如果使用大端机器, 为了使内存中的字节仍为72 6c 64 00, 应将i设置为: 0x726c6400
15 | 不需要修改57616, 因为整数的数值和十六进制输出不受字节序影响。
16 |
17 | 6.printf("x=%d y=%d", 3) 中只提供了一个整数参数, 但格式字符串要求两个。
18 | x会打印3, y的结果是不确定的。因为printf会按照格式字符串继续读取第二个整数参数, 但调用者没有提供它。它会把相应参数寄存器或栈中的残留内容当作参数。
```

Backtrace

要求:

为 xv6 内核实现 `backtrace()` 函数, 通过读取当前帧指针 `s0`, 沿内核栈逐层获取并打印各级函数的返回地址。将其加入 `sys_sleep()` 和 `panic()`, 并使用 `addr2line` 将返回地址转换为对应的源文件和代码行。从而理解 RISC-V 栈帧结构、帧指针、返回地址及内核调试方法。

解决:

1. 添加读取帧指针的函数

在 `kernel/riscv.h` 中添加:

```
1 | static inline uint64
2 | r_fp()
3 | {
4 |     uint64 x;
5 |     asm volatile("mv %0, s0" : "=r" (x));
6 |     return x;
7 | }
```

该函数通过内联汇编读取 RISC-V 的 `s0` 寄存器, 获得当前函数的帧指针。

2. 声明 `backtrace()`

在 `kernel/defs.h` 的 `printf.c` 声明区域添加：

```
1 | void backtrace(void);
```

3. 在内核中实现 `backtrace()`

在 `kernel/printf.c` 中添加函数，该函数首先获取当前帧指针，然后在当前内核栈页面范围内逐层读取栈帧。

```
1 | void backtrace(void)
2 | {
3 |     printf("backtrace:\n");
4 |
5 |     uint64 fp = r_fp();
6 |     uint64 stack_bottom = PGROUNDDOWN(fp);
7 |     uint64 stack_top = PGROUNDUP(fp);
8 |
9 |     while(fp >= stack_bottom + 16 && fp < stack_top){
10 |         uint64 ra = *((uint64 *) (fp - 8));
11 |         printf("%p\n", ra);
12 |         fp = *((uint64 *) (fp - 16));
13 |     }
14 | }
```

4. 在 `sys_sleep()` 中调用

根据提示，在 `kernel/sysproc.c` 的 `sys_sleep()` 开头添加：

```
1 | backtrace();
```

5. 在 `panic()` 中调用

根据提示，在确认 `bttest` 运行正确后，在 `kernel/printf.c` 的 `panic()` 中调用：

```
1 | backtrace();
```

Alarm

要求：

为 xv6 添加用户级定时报警功能，实现 `sigalarm(interval, handler)` 和 `sigreturn()` 系统调用。进程每运行指定数量的时钟周期后，内核自动调用用户设置的处理函数；处理完成后恢复被中断时的寄存器和执行位置，并继续运行原程序。同时需要支持周期触发、关闭报警，并防止处理函数被重复嵌套调用。实现系统调用、时钟中断、陷阱帧保存与恢复以及用户级中断处理。

解决：

1. 把测试程序加入编译

首先在 `Makefile` 的 `UPROGS` 下添加：

```
1 | $U/_alarmtest\
```

2. 在 `user/user.h` 声明系统调用

添加:

```
1 | int sigalarm(int ticks, void (*handler)());  
2 | int sigreturn(void);
```

其中 `handler` 是用户态报警处理函数的地址。

3. 在 `user/usys.pl` 添加系统调用桩

```
1 | entry("sigalarm");  
2 | entry("sigreturn");
```

编译时会自动生成汇编入口, 通过 `ecall` 从用户态进入内核。

4. 分配系统调用编号

在 `kernel/syscall.h` 中添加:

```
1 | #define SYS_sigalarm 22  
2 | #define SYS_sigreturn 23
```

5. 在内核中声明系统调用

在 `kernel/syscall.c` 中添加:

```
1 | extern uint64 sys_sigalarm(void);  
2 | extern uint64 sys_sigreturn(void);
```

6. 注册系统调用

在 `kernel/syscall.c` 的 `syscalls[]` 数组中添加:

```
1 | [SYS_sigalarm] sys_sigalarm,  
2 | [SYS_sigreturn] sys_sigreturn,
```

7. 在进程结构中保存报警状态

在 `kernel/proc.h` 的 `struct proc` 中添加:

```
1 | int alarm_interval;           //每隔多少次定时器中断触发  
2 | uint64 alarm_handler;        //用户处理函数的地址  
3 | int alarm_ticks;             //当前已经累计了多少次中断  
4 |  
5 | struct trapframe alarm_tf;   //保存中断发生时的全部用户寄存器  
6 | int alarm_handling;          //当前是否已经在处理alarm
```

8. 初始化报警字段

在 `kernel/proc.c` 的 `allocproc()` 中添加:

```
1 p->alarm_interval = 0;
2 p->alarm_handler = 0;
3 p->alarm_ticks = 0;
4 p->alarm_handling = 0;
```

新进程默认不启用定时报警。

9. 实现 `sys_sigalarm()`

在 `kernel/sysproc.c` 中添加:

```
1 uint64
2 sys_sigalarm(void)
3 {
4     int ticks;
5     uint64 handler;
6
7     argint(0,&ticks);
8     argaddr(1,&handler);
9
10    struct proc *p=myproc();
11
12    p->alarm_interval = ticks;
13    p->alarm_handler = handler;
14    p->alarm_ticks = 0;
15
16    return 0;
17 }
```

10. 在定时器中断中累计时钟周期

在 `kernel/trap.c` 的 `usertrap()` 中添加定时报警处理:

```
1 if(which_dev == 2)//本次陷阱来自时钟中断
2 {
3     if(p->alarm_interval > 0 && !p->alarm_handling)
4     {
5         p->alarm_ticks++;
6         if(p->alarm_ticks >= p->alarm_interval)
7         {
8             p->alarm_ticks = 0;
9             p->alarm_handling = 1;
10            //保存被中断程序的现场
11            memmove(&p->alarm_tf, p->trapframe, sizeof(struct trapframe));
12
13            p->trapframe->epc = p->alarm_handler;
14        }
15    }
16 }
```


其中不能使用：

```
1 | p->alarm_handler != 0
```

判断报警是否启用，因为用户处理函数地址可能恰好是 0。

11. 实现 `sys_sigreturn()`

在 `kernel/sysproc.c` 中添加：

```
1 | uint64 sys_sigreturn(void)
2 | {
3 |     struct proc *p = myproc();
4 |
5 |     memmove(p->trapframe, &p->alarm_tf, sizeof(struct trapframe));
6 |
7 |     p->alarm_handling = 0;
8 |
9 |     return p->trapframe->a0;
10 | }
```

该系统调用恢复报警发生前的完整用户现场，使程序从被中断的位置继续执行。

Lab总结

实验结果：

```
== Test answers-traps.txt == answers-traps.txt: OK
== Test backtrace test ==
$ make qemu-gdb
backtrace test: OK (2.1s)
== Test running alarmtest ==
$ make qemu-gdb
(3.2s)
== Test alarmtest: test0 ==
alarmtest: test0: OK
== Test alarmtest: test1 ==
alarmtest: test1: OK
== Test alarmtest: test2 ==
alarmtest: test2: OK
== Test usertests ==
$ make qemu-gdb
usertests: OK (46.2s)
== Test time ==
time: OK
Score: 85/85
lanqi@NIU:~/xv6-labs-2020$
```

问题与解决

1. RISC-V 汇编代码不容易理解

刚开始阅读 `user/call.asm` 时，对函数参数、返回地址以及函数调用过程不熟悉，难以确定 C 语言代码与汇编指令之间的对应关系。通过查阅，了解到 `a0~a7` 用于传递参数，其中 `a0` 和 `a1` 也用于保存返回值；`ra` 保存函数返回地址；`sp` 和 `s0` 分别作为栈指针和帧指针。同时结合 C 源码逐条分析 `call.asm`，理解了 `jal`、`jalr` 等指令的作用。对于没有找到的函数调用，则通过反汇编结果判断其可能已被编译器内联。

2. 报警函数执行后程序崩溃

最初只在定时器到期时将：

```
1 | p->trapframe->epc
```

修改为报警处理函数的地址。虽然能够打印 `alarm!`，但处理函数结束后无法恢复原程序，导致程序崩溃或测试失败。原因是报警处理函数会修改 `epc`、`ra`、`sp`、`a0` 等寄存器。解决方法是在跳转到处理函数之前备份整个 `trapframe`，而不是只保存 `epc`。当处理函数调用 `sigreturn()` 时，再恢复完整的陷阱帧，使程序能够从被中断的位置继续执行。

实验心得

通过本次实验，我对 xv6 从用户态进入内核态，再返回用户态的完整过程有了更加直观的认识。RISC-V 汇编练习让我理解了参数寄存器、返回地址等之间的关系，在实现 `backtrace()` 的过程中，我认识到高级语言中的函数调用在底层表现为严格的寄存器保存和栈帧组织。`Alarm` 实验则将系统调用、定时器中断和陷阱帧恢复联系到了一起。处理函数并不是普通的函数调用，而是内核通过修改 `epc` 强制改变用户程序的执行位置。要让程序在处理结束后继续正常运行，必须完整保存和恢复处理器现场，同时考虑周期触发和防止重入等问题。

Lab: xv6 lazy page allocation

Eliminate allocation from sbrk()

要求：

修改 xv6 的 `sbrk()` 系统调用，使其在扩大进程堆空间时只更新进程地址空间大小 `p->sz`，不立即分配物理内存或建立页表映射。随后运行程序并观察首次访问未映射内存时产生的缺页异常，为后续实现缺页处理和真正的惰性内存分配做准备。

解决：

1. 切换到 lazy 分支

```
1 | git checkout lazy
```

2. 修改 `sys_sbrk()`

在 `kernel/sysproc.c` 中删除立即分配逻辑，`sys_sbrk` 变为：

```
1 | uint64 sys_sbrk(void)
2 | {
3 |     uint64 addr;
4 |     int n;
5 |     struct proc *p = myproc();
```

```

6
7     if(argint(0, &n) < 0)
8         return -1;
9
10    addr = p->sz;
11    p->sz=p->sz+n;
12
13    return addr;
14 }

```

此时只是扩大了进程的逻辑地址范围，并没有为新区域分配物理页。

这时会出错，出错的原因是（就题目示例而言）：`sbrk()` 扩大了 `p->sz`，所以 `0x4008` 在进程申请的合法范围内；但没有为它分配物理页并建立页表映射，因此写入该地址时产生 `scause=15` 的 store page fault。

`stval=0x4008` 指明了发生缺页的虚拟地址，而 `sepc=0x1258` 指明了触发缺页的用户指令地址。

Lazy allocation

要求：

修改 xv6 的用户陷阱处理逻辑，使内核能够识别用户态产生的读、写缺页异常，并根据故障地址按需分配和清零物理页，再建立对应的页表映射，使进程返回用户态后继续执行。同时修改页面释放逻辑，使其能够跳过尚未映射的懒分配页面。实现缺页异常处理、物理页分配、虚拟地址对齐和页表映射。

解决：

在上一题的基础上进行修改：

1. 实现 `usertrap()` 的缺页处理

在 `kernel/trap.c` 中添加：

```

1  else if(r_scause()==15 || r_scause()==13)
2  {
3      //scause = 13: Load page fault, 读取未映射页面
4      //scause = 15: Store/AMO page fault, 写入未映射页面
5      uint64 va = PGROUNDDOWN(r_stval());
6
7      if(va >= p->sz || va >= MAXVA){
8          p->killed = 1;
9      }
10     else{
11         char *mem = kalloc();
12         if(mem == 0){
13             p->killed=1;
14         }
15         else{
16             memset(mem, 0, PGSIZE);
17             if(mappages(p->pagetable, va, PGSIZE, (uint64)mem, PTE_W|PTE_X|PTE_R|PTE_U)
18                 != 0){
19                 kfree(mem);
20                 p->killed=1;
21             }
22         }
23     }
24 }

```

```
22     }
23 }
```

实现识别缺页异常，分配物理页，建立用户页表映射等。

2. 修改 `uvmunmap()`

将 `kernel/vm.c` 的 `uvmunmap()` 中原来遇到未映射页面便 `panic` 的代码修改为跳过未映射页面：

```
1  if ((pte = walk(pagetable, a, 0)) == 0)
2      continue;
3  if ((*pte & PTE_V) == 0)
4      continue;
```

Lazytests and Ustests

要求：

完善 xv6 的懒分配机制，使其能够正确处理负数 `sbrk()`、非法地址缺页、用户栈保护页、`fork()` 中的父进程到子进程的内存复制、系统调用访问尚未分配的用户内存以及物理内存不足等情况。最终保证 `lazytests` 和 `ustests` 的所有测试均能通过。

解决：

这个实验主要是要处理一些特殊情况：

1. 修改 `sys_sbrk()` 使其支持负数参数

在 `kernel/sysproc.c` 中修改 `sys_sbrk()`：

```
1  if(n<0){
2      if(growproc(n)<0)
3          return -1;
4  }
5  else{
6      p->sz=p->sz+n;
7  }
```

其中：

- `n > 0`：只增大 `p->sz`，保持懒分配；
- `n < 0`：调用 `growproc()` 释放已经实际分配的页面。

2. 封装懒分配函数

在上一个实验中，把懒分配的代码直接写在 `usertrap()` 中，但是为了更加方便的使用，把它单独封装成一个函数。

在 `kernel/vm.c` 中加入懒分配实现函数，并在 `defs.h` 中添加声明：

```

uint64 lazyalloc(pagetable_t pagetable, uint64 va)
{
    struct proc *p = myproc();
    char *mem;
    pte_t *pte;

    //进程在高于sbrk()已分配地址的虚拟内存地址
    if(va >= MAXVA || va >= p->sz)
        return 0;

    //用户栈下方无效页面
    //sp是用户程序发生陷阱那一刻的用户栈指针
    if(va < PGROUNDDOWN(p->trapframe->sp))
        return 0;

    va = PGROUNDDOWN(va);

    mem = kalloc();
    if(mem == 0)
        return 0;

    memset(mem, 0, PGSIZE);

    if(mappages(p->pagetable, va, PGSIZE, (uint64)mem, PTE_W|PTE_X|PTE_R|PTE_U) != 0)
    {
        kfree(mem);
        return 0;
    }

    return (uint64)mem;
}

```

该函数负责：

- 检查地址是否超过 `sbrk()` 申请范围；
- 拒绝用户栈下方的保护页；
- 分配并清零物理页；
- 建立用户页表映射。

3. 简化 `usertrap()` 的缺页处理，让缺页处理调用 `lazyalloc()`

在 `kernel/trap.c` 的 `usertrap()` 中添加：

```

1  else if(r_scause()==15||r_scause()==13)
2  {
3      //scause = 13: Load page fault, 读取未映射页面
4      //scause = 15: Store/AMO page fault, 写入未映射页面
5      uint64 va = r_stval();
6
7      if(lazyalloc(p->pagetable, va)==0)
8          p->killed=1;
9  }

```

4. 修改 `uvmcopy()` 跳过页表空洞

在 `kernel/vm.c` 的 `uvmcopy()` 中, 将:

```

1  if((pte = walk(old, i, 0)) == 0)
2      panic("uvmcopy: pte should exist");
3  if((*pte & PTE_V) == 0)
4      panic("uvmcopy: page not present");

```

改为:

```

1  if((pte = walk(old, i, 0)) == 0)
2      continue;
3  if((*pte & PTE_V) == 0)
4      continue;

```

这样 `fork()` 时:

- 已经分配的页面正常复制;
- 尚未分配的懒分配页面继续保持为空;
- 子进程首次访问空洞时自行触发缺页异常。

5. 修改 `copyout()` 支持懒分配地址

在 `kernel/vm.c` 的 `copyout()` 改为:

```

1  pa0 = walkaddr(pagetable, va0);
2
3  if (pa0 == 0) {
4      if (lazyalloc(pagetable, va0) == 0)
5          return -1;
6
7      pa0 = walkaddr(pagetable, va0);
8      if (pa0 == 0)
9          return -1;
10 }

```

这样 `read()` 可以把数据写入由 `sbrk()` 申请但尚未实际分配的用户缓冲区。

6. 修改 `copyin()` 支持懒分配地址

`copyin()` 改为:

```
1 pa0 = walkaddr(pagetable, va0);
2
3 if (pa0 == 0) {
4     if (lazyalloc(pagetable, va0) == 0)
5         return -1;
6
7     pa0 = walkaddr(pagetable, va0);
8     if (pa0 == 0)
9         return -1;
10 }
```

这样内核可以从合法但尚未分配的用户页面读取数据。

7. 修改 `copyinstr()` 支持懒分配地址

`copyinstr()` 改为:

```
1 pa0 = walkaddr(pagetable, va0);
2
3 if (pa0 == 0) {
4     if (lazyalloc(pagetable, va0) == 0)
5         return -1;
6
7     pa0 = walkaddr(pagetable, va0);
8     if (pa0 == 0)
9         return -1;
10 }
```

这样系统调用读取用户字符串时，也能处理尚未实际分配的合法地址。

Lab总结

实验结果:

```

usertests: opentest: OK
== Test  usertests: writetest ==
usertests: writetest: OK
== Test  usertests: writebig ==
usertests: writebig: OK
== Test  usertests: createtest ==
usertests: createtest: OK
== Test  usertests: openiput ==
usertests: openiput: OK
== Test  usertests: exitiput ==
usertests: exitiput: OK
== Test  usertests: iput ==
usertests: iput: OK
== Test  usertests: mem ==
usertests: mem: OK
== Test  usertests: pipe1 ==
usertests: pipe1: OK
== Test  usertests: preempt ==
usertests: preempt: OK
== Test  usertests: exitwait ==
usertests: exitwait: OK
== Test  usertests: rmdot ==
usertests: rmdot: OK
== Test  usertests: fourteen ==
usertests: fourteen: OK
== Test  usertests: bigfile ==
usertests: bigfile: OK
== Test  usertests: dirfile ==
usertests: dirfile: OK
== Test  usertests: iref ==
usertests: iref: OK
== Test  usertests: forktest ==
usertests: forktest: OK
== Test time ==
time: OK
Score: 119/119
lanqi@NIU:~/xv6-labs-2020$

```

问题与解决

1. `fork()` 遇到未映射页面时失败

原来的 `uvmcopy()` 假定从地址0到 `p->sz` 的所有页面都存在映射。懒分配后，地址空间中可能存在空洞，导致 `fork()` 时出现 panic。为了解决这个问题，修改 `uvmcopy()`，遇到不存在或无效的 PTE 时直接跳过。父进程已经分配的页面正常复制，未分配的页面在子进程中继续保持未映射，等子进程真正访问时再分配。

2. 系统调用无法访问懒分配内存

用户程序可能将 `sbrk()` 返回的地址直接传给 `read()` 或 `write()`。此时页面尚未映射，而访问发生在内核的 `copyin()`、`copyout()` 或 `copyinstr()` 中，不会经过用户态的 `usertrap()`。为了解决这个问题，在这些复制函数中，当 `walkaddr()` 找不到映射时，主动调用懒分配函数建立页面映射，然后重新查询物理地址并完成数据复制。

实验心得

通过本次实验，我理解了懒分配并不是简单地推迟一次 `ka1loc()`，而是会改变内核对进程地址空间的基本假设。以前 `p->sz` 范围内的页面默认都已经映射；引入懒分配后，合法地址空间中允许存在尚未映射的空洞，因此页表复制、页面释放和内核访问用户内存的逻辑都需要同步调整。

本实验也让我更清楚地理解了缺页异常的处理过程：CPU 保存故障指令和地址，内核根据 `scause` 与 `stval` 判断异常原因，分配并映射页面后保持 `epc` 不变，使原指令重新执行。这个过程体现了虚拟内存硬件和操作系统内核之间的配合。

此外，实验说明了地址合法与页面已映射是两个不同概念。一个地址可以位于 `p->sz` 范围内，但尚未对应物理内存；也可能发生缺页异常，却属于越界访问或栈保护页，不能进行分配。只有正确完成边界判断、错误回收和内存不足处理，懒分配机制才能安全运行。

Lab: Copy-on-Write Fork for xv6

Implement copy-on write

要求：

为 xv6 实现写时复制（Copy-on-Write, COW）版本的 `fork()`。父子进程创建时共享相同的物理页面，并将相关页表项设为只读和 COW；当任一进程尝试写入共享页面时，内核通过缺页异常分配并复制新页面。还需使用引用计数管理共享物理页，并让 `copyout()` 正确处理 COW 页面，最终通过 `cowtest` 和 `usertests`。

解决：

1. 切换到 `cow` 分支

```
1 | git checkout cow
```

2. 添加 COW 页表标志

在 `kernel/riscv.h` 的 PTE 标志区域添加：

```
1 | #define PTE_COW (1L << 8)
```

因为第8位属于 RISC-V 页表项中的软件保留位，可由内核用于标记 COW 页面。

3. 为物理页添加引用计数

在 `kernel/kalloc.c` 中添加：

```
1 | struct {
2 |     struct spinlock lock;
3 |     int count[PHYSTOP / PGSIZE];
4 | } refcnt;
```

引用计数用于记录每个物理页当前被多少个页表引用。

4. 初始化引用计数锁

在 `kinit()` 中添加：

```
1 | initlock(&refcnt.lock, "refcnt");
```

5. 修改空闲内存初始化

在 `freerange()` 中, 调用 `kfree()` 前先将引用计数设置为1:

```
1 for (; p + PGSIZE <= (char *)pa_end;
2     p += PGSIZE) {
3     refcnt.count[(uint64)p / PGSIZE] = 1;
4     kfree(p);
5 }
```

因为修改后的 `kfree()` 会先将引用计数减1, 使初始化页面的计数从1变为0, 再放入空闲链表。

6. 修改 `kalloc()`

在 `kernel/kalloc.c` 的 `kalloc()` 中, 将新分配页面的引用计数设置为1:

```
1 if (r) {
2     memset((char *)r, 5, PGSIZE);
3
4     acquire(&refcnt.lock);
5     refcnt.count[(uint64)r / PGSIZE] = 1;
6     release(&refcnt.lock);
7 }
```

7. 修改 `kfree()`

让 `kfree()` 先减少引用计数, 只有计数变成0时才真正释放页面:

```
void
kfree(void *pa)
{
    struct run *r;

    if(((uint64)pa % PGSIZE) != 0 || (char*)pa < end || (uint64)pa >= PHYSTOP)
        panic("kfree");

    acquire(&refcnt.lock);
    refcnt.count[(uint64)pa / PGSIZE]--;
    int refs = refcnt.count[(uint64)pa / PGSIZE];
    release(&refcnt.lock);

    if(refs > 0) // 引用计数仍大于0, 直接返回
        return;

    if(refs < 0)
        panic("kfree: negative reference");

    // Fill with junk to catch dangling refs.
    memset(pa, 1, PGSIZE);

    r = (struct run*)pa;

    acquire(&kmem.lock);
    r->next = kmem.freelist;
    kmem.freelist = r;
    release(&kmem.lock);
}
```

8. 添加增加引用计数的函数

在 `kernel/kalloc.c` 中添加:

```
1 //增加引用计数的函数
2 void kaddref(uint64 pa)
3 {
4     if(pa >= PHYSTOP || pa % PGSIZE != 0)
5         panic("kaddref");
6
7     acquire(&refcnt.lock);
8     refcnt.count[pa / PGSIZE]++;
9     release(&refcnt.lock);
10 }
```

并在 `kernel/defs.h` 中声明:

```
1 void kaddref(uint64);
```

9. 修改 `uvmcopy()` 共享物理页

在 `kernel/vm.c` 中修改 `uvmcopy()`, 取消为子进程分配和复制物理页:

```

//将父进程的物理页面映射到子进程中
int uvmcopy(pagetable_t old, pagetable_t new, uint64 sz)
{
    pte_t *pte;
    uint64 pa, i;
    uint flags;

    for(i = 0; i < sz; i += PGSIZE){
        if((pte = walk(old, i, 0)) == 0)
            panic("uvmcopy: pte should exist");
        if((*pte & PTE_V) == 0)
            panic("uvmcopy: page not present");
        pa = PTE2PA(*pte); //页面的地址
        flags = PTE_FLAGS(*pte); //页面的标志

        if(flags & PTE_W){
            flags = (flags & ~PTE_W) | PTE_COW; //清除写权限，增加cow标志
            *pte = PA2PTE(pa) | flags;
        }
        //在 new 页表中：虚拟地址 i  ——> 物理地址 pa，映射长度：PGSIZE，权限标志：flags
        if(mappages(new, i, PGSIZE, pa, flags) != 0)
            goto err;

        //引用计数增加1
        kaddref(pa);
    }
    //刷新TLB
    sfence_vma();
    return 0;

err:
    uvmunmap(new, 0, i / PGSIZE, 1);
    //刷新TLB
    sfence_vma();
    return -1;
}

```

这会：

- 让父子进程共享相同物理页；
- 清除原本可写页面的 `PTE_W`；
- 为其添加 `PTE_COW`；
- 增加共享物理页的引用计数。

10. 声明 COW 缺页处理函数

在 `kernel/defs.h` 的 `vm.c` 声明区域添加：

```
1 | int cowalloc(pagetable_t, uint64);
```

11. 实现 `cowalloc()`

在 `kernel/vm.c` 中添加：

```

int cowalloc(pagetable_t pagetable, uint64 va)
{
    pte_t *pte;
    uint64 pa;
    uint flags;
    char *mem;
    //向下对齐
    va = PGROUNDDOWN(va);
    //超过MAXVA的地址
    if(va >= MAXVA)
        return -1;
    pte = walk(pagetable, va, 0);
    if(pte == 0)
        return -1;
    //页面有效，用户可以访问，页面确实是 COW 页面
    if((*pte & PTE_V) == 0 || (*pte & PTE_U) == 0 || (*pte & PTE_COW) == 0)
        return -1;
    pa = PTE2PA(*pte);
    flags = PTE_FLAGS(*pte);
    //为当前进程分配自己的物理页。
    mem = kalloc();
    if(mem == 0)
        return -1;
    //将共享物理页的内容复制到新页。
    memmove(mem, (char*)pa, PGSIZE);
    //当前进程的页表改为指向新物理页，增加写权限，清除 COW 标志，其余标志保持不变
    flags = (flags | PTE_W) & ~PTE_COW;
    *pte = PA2PTE((uint64)mem) | flags;
    //当前进程不再引用旧页，kfree减少引用计数
    kfree((void*)pa);
    //页表已经变化，清除旧的 TLB 缓存
    sfence_vma();
    return 0;
}

```

该函数会为发生写异常的进程分配新物理页，复制原页面内容，并将新页面设置为可写。

12. 在 `usertrap()` 中处理 COW 写异常

在 `kernel/trap.c` 的 `usertrap()` 中添加：

```

1  else if(r_scause() == 15){
2      uint64 va = r_stval();
3      if(va >= p->sz || cowalloc(p->pagetable, va) < 0)
4          p->killed = 1;
5  }

```

`scause == 15` 表示 Store/AMO page fault。

如果地址非法、页面不是 COW 页面或新页面分配失败，则终止当前进程。

13. 修改 `copyout()` 处理 COW 页面

修改 `kernel/vm.c` 的 `copyout()`，因为 `copyout()` 在内核中直接写物理内存，不会触发用户态写异常，因此必须主动处理 COW 页面。

```

int copyout(pagetable_t pagetable, uint64 dstva, char *src, uint64 len)
{
    uint64 n, va0, pa0;
    pte_t *pte;

    while(len > 0){

        if(dstva >= MAXVA)
            return -1;
        //获取当前目标地址所在页面的起始虚拟地址
        va0 = PGROUNDDOWN(dstva);
        //找到目标页面的页表项
        pte = walk(pagetable, va0, 0);
        if(pte == 0)
            return -1;
        //copyout()由内核执行，不会自动产生用户页错误，目标是COW页面，需要主动复制。
        if((*pte & PTE_COW) != 0){
            if(cowalloc(pagetable, va0) < 0)
                return -1;
        }
        //cowalloc()可能修改了PTE，重新取得物理地址。
        pa0 = walkaddr(pagetable, va0);
        if(pa0 == 0)
            return -1;

        n = PGSIZE - (dstva - va0);
        if(n > len)
            n = len;
        memmove((void *) (pa0 + (dstva - va0)), src, n);

        len -= n;
        src += n;
        dstva = va0 + PGSIZE;
    }
    return 0;
}

```

Lab总结

实验结果：

```
== Test running cowtest ==
$ make qemu-gdb
(4.4s)
== Test    simple ==
    simple: OK
== Test    three ==
    three: OK
== Test    file ==
    file: OK
== Test usertests ==
$ make qemu-gdb
(49.5s)
== Test    usertests: copyin ==
    usertests: copyin: OK
== Test    usertests: copyout ==
    usertests: copyout: OK
== Test    usertests: all tests ==
    usertests: all tests: OK
== Test time ==
time: OK
Score: 110/110
```

问题与解决

1. 修改 PTE 后仍使用旧权限

修改父进程页表项后，CPU 的 TLB 中可能仍保存旧的可写映射，导致写操作不触发 COW 缺页异常。为了解决这一问题，修改页表项后调用 `sfence_vma()`，刷新 TLB 中缓存的地址转换结果。

2. `copyout()` 破坏父子进程的数据隔离

`copyout()` 在内核态直接向用户物理页写入数据，不会触发用户态缺页异常。如果目标是 COW 页面，内核可能直接修改父子进程共享的物理页。为了解决这个问题，在 `copyout()` 中检查目标 PTE 是否带有 `PTE_COW`。如果是，先调用 COW 处理函数完成页面复制，再向新的物理页写入数据。

然而，`copyin()` 和 `copyinstr()` 只读取用户页面，不会修改共享数据，因此不需要进行 COW 复制。COW 的基本原则是：共享读取是安全的，共享写入必须先复制。

实验心得

通过本次实验，我进一步理解了虚拟内存和页表提供的间接映射能力。写时复制并不是在 `fork()` 时立即复制进程内存，而是先让父子进程共享物理页，只有某个进程真正写入时才分配并复制页面。这种方式减少了不必要的内存分配和数据复制，尤其适合子进程随后立即调用 `exec()` 的情况。

实验也让我认识到，COW 并不只是修改 `uvmcopy()`。一个完整实现还需要同时考虑页表权限、缺页异常、TLB 刷新、物理页引用计数、进程退出以及 `copyout()` 等内核写用户内存的特殊路径。任何一个环节处理不当，都可能造成数据相互影响、内存泄漏、重复释放或内核崩溃。此外，引用计数让我更清楚地理解了共享资源的生命周期管理：页面是否能够释放，不能只取决于当前进程是否结束，而应取决于是否仍有其他页表引用它。

Lab: Multithreading

Uthread: switching between threads

要求:

完善 xv6 的用户级线程库，实现线程创建和线程上下文切换。每个线程需要拥有独立的栈和寄存器上下文；调度时保存当前线程的被调用者保存寄存器，并恢复下一个线程的寄存器，使线程能够从上次暂停的位置继续运行。实现线程调度、RISC-V 调用约定、栈初始化和汇编上下文切换。

解决:

1. 切换到 `thread` 分支

```
1 | git checkout thread
```

2. 为线程添加寄存器上下文

在 `user/uthread.c` 中添加:

```
//线程上下文结构
struct thread_context {
    uint64 ra;
    uint64 sp;

    uint64 s0;
    uint64 s1;
    uint64 s2;
    uint64 s3;
    uint64 s4;
    uint64 s5;
    uint64 s6;
    uint64 s7;
    uint64 s8;
    uint64 s9;
    uint64 s10;
    uint64 s11;
};

struct thread {
    char    stack[STACK_SIZE]; /* the thread's stack */
    int     state;              /* FREE, RUNNING, RUNNABLE */
    struct thread_context context;
};
```

3. 声明线程切换函数

在 `user/uthread.c` 中修改声明，便于理解，第一个参数表示需要保存的旧线程上下文，第二个参数表示需要恢复的新线程上下文。

```
1 | extern void thread_switch(struct thread_context *, struct thread_context *);
```


4. 初始化新线程上下文

在 `user/uthread.c` 的 `thread_create` 中添加：

```
1 | t->context.ra = (uint64)func;
2 | t->context.sp = (uint64)t->stack + STACK_SIZE;
```

其中：

- `ra` 指向线程入口函数；
- `sp` 指向线程独立栈的顶部。

新线程第一次被调度时，`thread_switch()` 最后的 `ret` 会跳转到 `func()`。

5. 在调度器中切换线程

在 `thread_schedule()` 中添加：

```
1 | thread_switch(&t->context, &next_thread->context);
```

6. 实现汇编上下文切换

在 `user/uthread_switch.S` 中模仿 `swtch.S` 添加：

```

8
9     .globl thread_switch
10    thread_switch:
11        sd ra, 0(a0)
12        sd sp, 8(a0)
13        sd s0, 16(a0)
14        sd s1, 24(a0)
15        sd s2, 32(a0)
16        sd s3, 40(a0)
17        sd s4, 48(a0)
18        sd s5, 56(a0)
19        sd s6, 64(a0)
20        sd s7, 72(a0)
21        sd s8, 80(a0)
22        sd s9, 88(a0)
23        sd s10, 96(a0)
24        sd s11, 104(a0)
25
26        ld ra, 0(a1)
27        ld sp, 8(a1)
28        ld s0, 16(a1)
29        ld s1, 24(a1)
30        ld s2, 32(a1)
31        ld s3, 40(a1)
32        ld s4, 48(a1)
33        ld s5, 56(a1)
34        ld s6, 64(a1)
35        ld s7, 72(a1)
36        ld s8, 80(a1)
37        ld s9, 88(a1)
38        ld s10, 96(a1)
39        ld s11, 104(a1)
40
41        ret    /* return to ra */
42

```

Using threads

要求:

这个题目要求使用 Linux/macOS 的 POSIX 线程和互斥锁修复多线程哈希表中的竞态条件，分析并记录并发插入导致键丢失的原因。先通过加锁保证两线程运行时没有键丢失，再为每个哈希桶设置独立锁，使不同桶的插入操作能够并行执行，在保证正确性的同时获得至少 1.25 倍的并行加速。最终通过 `ph_safe` 和 `ph_fast` 测试。

解决:

1. 运行原始程序

在 xv6 项目根目录执行：

```

1  make ph
2  ./ph 1
3  ./ph 2

```

单线程没有键丢失，而双线程会出现 keys missing：

```
lanqi@NIU:~/xv6-labs-2020$ make ph
gcc -o ph -g -O2 notxv6/ph.c -pthread
lanqi@NIU:~/xv6-labs-2020$ ./ph 1
100000 puts, 3.962 seconds, 25240 puts/second
0: 0 keys missing
100000 gets, 3.719 seconds, 26888 gets/second
lanqi@NIU:~/xv6-labs-2020$ ./ph 2
100000 puts, 1.626 seconds, 61490 puts/second
0: 16565 keys missing
1: 16565 keys missing
200000 gets, 6.050 seconds, 33060 gets/second
```

在项目根目录创建并填写 answers-thread.txt：

如果使用两个线程时，两个线程可能同时向哈希表的同一个桶中插入元素，从而发生竞态条件并造成更新丢失。假设 table[i] 原来指向节点 E。线程 1 要插入节点 A，线程 2 要插入节点 B，可能发生以下执行顺序：

1. 线程 1 读取 table[i]，得到 E。
2. 线程 2 也读取 table[i]，得到 E。
3. 线程 1 设置 A->next = E。
4. 线程 2 设置 B->next = E。
5. 线程 1 设置 table[i] = A。
6. 线程 2 设置 table[i] = B。

最后链表变成 table[i] -> B -> E，而节点 A 不再能从 table[i] 访问到。线程 2 对桶头指针的写入覆盖了线程 1 的写入，因此 A 对应的 key 丢失。

而使用一个线程时，插入操作按顺序执行，不会出现两个线程同时读取和修改同一个桶头指针的情况，因此不会发生这种更新覆盖，也就不会丢失 key。

2. 先添加全局锁保证正确性

在 notxv6/ph.c 的全局变量区域添加：

```
1 pthread_mutex_t lock;
```

在 main() 创建线程之前初始化：

```
1 pthread_mutex_init(&lock, NULL);
```

使用全局锁保护 put()：

在 put() 的查找和修改操作外加锁，锁覆盖查找、判断和插入的完整过程。

```
1 pthread_mutex_lock(&lock);
2 pthread_mutex_unlock(&lock);
```

使用全局锁保护 get()

在 get() 中添加：

```
1 pthread_mutex_lock(&lock);
2 pthread_mutex_unlock(&lock);
```

此时运行：

```
1 make ph
2 ./ph 1
3 ./ph 2
```

显示：

```
lanqi@NIU:~/xv6-labs-2020$ make ph
gcc -o ph -g -O2 notxv6/ph.c -pthread
lanqi@NIU:~/xv6-labs-2020$ ./ph 1
100000 puts, 5.642 seconds, 17723 puts/second
0: 0 keys missing
100000 gets, 4.029 seconds, 24817 gets/second
lanqi@NIU:~/xv6-labs-2020$ ./ph 2
100000 puts, 4.457 seconds, 22438 puts/second
0: 0 keys missing
1: 0 keys missing
200000 gets, 9.002 seconds, 22216 gets/second
```

全局锁可以保证正确性，但所有线程都竞争同一把锁，所以无法通过 `ph_fast` 性能测试。

```
== Test ph_safe == make[1]: Entering directory '/home/lanqi/xv6-labs-2020'
make[1]: 'ph' is up to date.
make[1]: Leaving directory '/home/lanqi/xv6-labs-2020'
ph_safe: OK (13.4s)
== Test ph_fast == make[1]: Entering directory '/home/lanqi/xv6-labs-2020'
make[1]: 'ph' is up to date.
make[1]: Leaving directory '/home/lanqi/xv6-labs-2020'
ph_fast: FAIL (23.2s)
Parallel put() speedup is less than 1.25x
```

3. 将全局锁改为桶锁数组

删除：

```
1 pthread_mutex_t lock;
```

改为：

```
1 pthread_mutex_t locks[NBUCKET];
```

每个哈希桶使用一把独立的互斥锁。

初始化所有桶锁

在 `main()` 创建线程之前添加：

```

1  for (int i = 0; i < NBUCKET; i++){
2      pthread_mutex_init(&locks[i], NULL);
3  }

```

使用桶锁保护 `put()`:

将全局锁替换为当前桶对应的锁:

```

1  pthread_mutex_lock(&locks[i]);
2  pthread_mutex_unlock(&locks[i]);

```

访问同一个桶的线程会互斥，而访问不同桶的线程可以并行运行。

使用桶锁保护 `get()`:

在 `get()` 中使用相同的桶锁:

```

1  pthread_mutex_lock(&locks[i]);
2  pthread_mutex_unlock(&locks[i]);

```

`put()` 和 `get()` 必须对同一个桶使用同一把锁。

再次测试:

```

• lanqi@NIU:~/xv6-labs-2020$ make ph
gcc -o ph -g -O2 notxv6/ph.c -pthread
• lanqi@NIU:~/xv6-labs-2020$ ./ph 1
100000 puts, 4.061 seconds, 24624 puts/second
0: 0 keys missing
100000 gets, 3.842 seconds, 26029 gets/second
• lanqi@NIU:~/xv6-labs-2020$ ./ph 2
100000 puts, 2.389 seconds, 41864 puts/second
1: 0 keys missing
0: 0 keys missing
200000 gets, 6.709 seconds, 29809 gets/second

```

```

== Test ph_safe == make[1]: Entering directory '/home/lanqi/xv6-labs-2020'
gcc -o ph -g -O2 notxv6/ph.c -pthread
make[1]: Leaving directory '/home/lanqi/xv6-labs-2020'
ph_safe: OK (7.5s)
== Test ph_fast == make[1]: Entering directory '/home/lanqi/xv6-labs-2020'
make[1]: 'ph' is up to date.
make[1]: Leaving directory '/home/lanqi/xv6-labs-2020'
ph_fast: OK (14.8s)

```

Barrier

要求:

使用互斥锁和条件变量实现可重复使用的线程屏障，使每一轮中所有参与线程都到达屏障后才能继续执行。需要正确维护到达线程数和轮次，使用 `pthread_cond_wait()` 阻塞未满足条件的线程，并使用

`pthread_cond_broadcast()` 唤醒所有等待线程，同时避免不同轮次之间相互干扰，最终通过 `barrier` 测试。

解决:

1. 确认屏障初始化

模板提供的 `barrier_init()` 中有：

```
1 static void
2 barrier_init(void)
3 {
4     assert(pthread_mutex_init(&bstate.barrier_mutex, NULL) == 0); //初始化互斥锁
5     assert(pthread_cond_init(&bstate.barrier_cond, NULL) == 0); //初始化条件变量
6     bstate.nthread = 0;
7 }
```

互斥锁、条件变量、到达人数和轮次必须在创建线程之前初始化。

2. 补全 `barrier()`

`notxv6/barrier.c` 的 `barrier()` 改为：

```
static void
barrier(void)
{
    pthread_mutex_lock(&bstate.barrier_mutex);

    //这个线程进入barrier时的轮次
    int round = bstate.round;

    //这个线程已经到达当前轮的屏障
    bstate.nthread++;

    if (bstate.nthread == nthread) {
        //这个线程是当前轮最后一个到达的线程。重置计数器并进入下一轮，然后唤醒所有等待线程。
        bstate.nthread = 0;
        bstate.round++;

        pthread_cond_broadcast(&bstate.barrier_cond);
    } else {
        //不是最后一个线程。
        //只要 round 没有改变，就说明当前轮尚未结束
        while (round == bstate.round) {
            pthread_cond_wait(&bstate.barrier_cond, &bstate.barrier_mutex);
        }
    }

    pthread_mutex_unlock(&bstate.barrier_mutex);
}
```

Lab总结

实验结果：

```

== Test uthread ==
$ make qemu-gdb
uthread: OK (2.4s)
== Test answers-thread.txt == answers-thread.txt: OK
== Test ph_safe == make[1]: Entering directory '/home/lanqi/xv6-labs-2020'
gcc -o ph -g -O2 notxv6/ph.c -pthread
make[1]: Leaving directory '/home/lanqi/xv6-labs-2020'
ph_safe: OK (7.5s)
== Test ph_fast == make[1]: Entering directory '/home/lanqi/xv6-labs-2020'
make[1]: 'ph' is up to date.
make[1]: Leaving directory '/home/lanqi/xv6-labs-2020'
ph_fast: OK (14.8s)
== Test barrier == make[1]: Entering directory '/home/lanqi/xv6-labs-2020'
gcc -o barrier -g -O2 notxv6/barrier.c -pthread
make[1]: Leaving directory '/home/lanqi/xv6-labs-2020'
barrier: OK (4.4s)
== Test time ==
time: OK
Score: 60/60

```

问题与解决

1. Uthread：用户级线程切换

在实现用户级线程时，最初线程无法正常启动或在切换几次后发生异常。主要原因是新线程没有正确初始化执行入口和独立栈，同时线程切换时保存的寄存器不完整。

解决方法是为每个线程增加独立的上下文结构。线程切换时需要保存 `ra`、`sp`、`s0~s11`，不需要保存 `a0~a7`、`t0~t6`，因为根据 RISC-V 调用约定，`a` 和 `t` 寄存器属于调用者保存寄存器，调用者本来就不能假设它们在函数调用后保持不变；`s0~s11` 属于被调用者保存寄存器，切换函数必须保存和恢复它们。除此之外，还必须保存 `ra` 和 `sp`，否则线程无法恢复原来的执行位置和独立栈。

创建线程时，将 `ra` 设置为线程函数地址，将 `sp` 设置为线程栈顶；切换线程时，在汇编函数中保存旧线程上下文并恢复新线程上下文。

2. 线程屏障部分线程提前进入下一轮

屏障的主要问题是部分线程会提前进入下一轮，导致不同线程的循环轮次不一致并触发断言错误。只使用到达线程计数器仍然无法区分前后两轮，快速线程可能在其他线程尚未完全离开上一轮时修改下一轮的计数。

解决方法是使用互斥锁保护共享状态，并同时维护 `nthread` 和 `round`。线程进入屏障时保存当前轮次并增加到达人数；最后一个到达的线程清零计数器、增加轮次，并使用 `pthread_cond_broadcast()` 唤醒所有等待线程；其他线程通过 `pthread_cond_wait()` 等待轮次发生变化。

实验心得

通过本次实验，我进一步理解了线程创建、上下文切换、互斥同步和条件变量的工作原理。用户级线程切换让我认识到，线程的运行状态本质上由寄存器上下文和栈共同决定，恢复 `ra` 和 `sp` 就能够让线程从原来的位置继续执行。哈希表实验说明，多线程程序即使单线程运行完全正确，在并发环境下也可能因为操作交错产生竞态条件。加锁不仅要考虑正确性，还需要合理控制锁的粒度。全局锁实现简单但限制并行性，而分桶锁能在保证数据安全的同时提高性能。屏障实验让我进一步理解了互斥锁和条件变量。

总体而言，本次实验让我认识到，并发程序的难点不仅是让多个线程同时运行，更重要的是正确管理共享状态、线程执行顺序和同步范围。

Lab: locks

Memory allocator

要求:

重新设计 xv6 的物理内存分配器，为每个 CPU 建立独立的空闲页链表和锁，使不同 CPU 能够并行执行 `kalloc()` 和 `kfree()`，减少对单一 `kmem` 锁的竞争。当当前 CPU 的空闲链表为空时，需要从其他 CPU 的链表中窃取空闲页。所有相关锁名必须以 `kmem` 开头，最终通过 `kalloc_test`、`user_test` `sbrkmuch` 和完整的 `user_test`。

解决:

1. 切换到 `lock` 分支

```
1 | git checkout lock
```

2. 将全局空闲链表改为每 CPU 一条链表

在 `kernel/kalloc.c` 中，将：

```
1 | struct {
2 |     struct spinlock lock;
3 |     struct run *freelist;
4 | } kmem;
```

修改为：

```
1 | struct {
2 |     struct spinlock lock;
3 |     struct run *freelist;
4 | } kmem[NCPU];
```

3. 初始化每个 CPU 的锁和空闲链表

修改 `kinit()`：

```
1 | void kinit()
2 | {
3 |     for(int i=0;i<NCPU;i++){
4 |         initlock(&kmem[i].lock, "kmem");
5 |         kmem[i].freelist = 0;
6 |     }
7 |     freerange(end, (void*)PHYSTOP); //通过kfree()把初始空闲页加入当前CPU的链表
8 | }
```

4. 在 `kfree()` 中获取当前 CPU 编号

根据提示，由于 `cpuid()` 只能在关闭中断时安全使用，因此通过：

```
1 | push_off();
2 | int id = cpuid();
```


获得当前 CPU 编号，操作完成后再调用：

```
1 pop_off();
```

5. 修改 kfree()

将释放的物理页加入当前 CPU 的空闲链表：

```
1 //释放页时，页会进入当前CPU的空闲链表
2 push_off();
3 int id = cpuid();
4
5 acquire(&kmem[id].lock);
6 r->next = kmem[id].freelist;
7 kmem[id].freelist = r;
8 release(&kmem[id].lock);
9
10 pop_off();
```

6. 实现偷页

在 kernel/kalloc.c 中修改 kalloc()：

先从当前 CPU 的链表获取页面；如果链表为空，则从其他 CPU 偷取：

```
void * kalloc(void)
{
    struct run *r;

    push_off();
    int id = cpuid();

    acquire(&kmem[id].lock);
    r = kmem[id].freelist;
    if(r)
        kmem[id].freelist = r->next;

    release(&kmem[id].lock);

    //当前 CPU 没有空闲页时，从其他 CPU 偷一个
    if(r == 0){
        for(int i = 0; i < NCPU; i++){
            if (i == id)
                continue;

            acquire(&kmem[i].lock);
            r = kmem[i].freelist;
            if (r)
                kmem[i].freelist = r->next;
            release(&kmem[i].lock);

            if (r)
                break;
        }
    }
    pop_off();

    if(r)
        memset((char*)r, 5, PGSIZE); // fill with junk
    return (void*)r;
}
```

Buffer cache

要求:

重新设计 xv6 的磁盘块缓存，使用哈希表组织缓存块，并为每个哈希桶设置独立锁，使不同磁盘块的查找和释放能够并行执行，降低 `bcache.lock` 的竞争。同时需要保证同一磁盘块在缓存中最多只有一个副本，并正确处理缓存未命中、LRU 缓存替换、跨桶移动和多锁死锁问题。所有相关锁名必须以 `bcache` 开头，最终通过 `bcachetest`、`usertests` 和 `make grade`。

解决:

1. 在缓冲区结构中添加时间戳和桶编号

修改 `kernel/buf.h` 的 `struct buf` :

```
1 struct buf {
2     int valid;    // has data been read from disk?
3     int disk;    // does disk "own" buf?
4     uint dev;    // 磁盘设备编号
5     uint blockno; // 磁盘块编号
6     struct sleeplock lock;
7     uint refcnt;  // 有多少使用者在使用该缓冲区
8     struct buf *prev; // LRU cache list
9     struct buf *next;
10    uchar data[BSIZE];
11    uint timestamp; // refcnt变成0时记录最后使用时间
12    int bucket;    // 桶号
13 };
```

2. 将全局缓存链表改为哈希桶

在 `kernel/bio.c` 中定义13个哈希桶:

```
1 #define NBUCKET 13
2
3 struct bucket{
4     struct spinlock lock;
5     struct buf head;
6 };
```

然后修改 `bcache` :

```
1 struct {
2     // 串行化缓存未命中后的淘汰过程
3     struct spinlock lock;
4     struct buf buf[NBUF];
5
6     struct bucket buckets[NBUCKET];
7 } bcache;
```

3. 声明系统时钟并添加哈希函数

在 `kernel/bio.c` 中添加:

```
1 | extern uint ticks;
```

然后添加:

```
1 | static int
2 | bhash(uint blockno)
3 | {
4 |     return blockno % NBUCKET;
5 | }
```

4. 添加桶链表操作函数

在 `kernel/bio.c` 中添加插入函数:

```
1 | static void
2 | binsert(struct bucket *bucket, struct buf *b)
3 | {
4 |     b->next = bucket->head.next;
5 |     b->prev = &bucket->head;
6 |
7 |     bucket->head.next->prev = b;
8 |     bucket->head.next = b;
9 | }
```

再添加删除函数:

```
1 | static void
2 | bremove(struct buf *b)
3 | {
4 |     b->prev->next = b->next;
5 |     b->next->prev = b->prev;
6 | }
```

5. 修改 `binit()`

初始化全局淘汰锁和所有哈希桶:

```

void
binit(void)
{
    struct buf *b;

    initlock(&bcache.lock, "bcache");

    for (int i = 0; i < NBUCKET; i++) {
        initlock(&bcache.buckets[i].lock, "bcache.bucket");
        //空桶
        //head.next == &head
        //head.prev == &head
        bcache.buckets[i].head.prev = &bcache.buckets[i].head;
        bcache.buckets[i].head.next = &bcache.buckets[i].head;
    }

    for(int i=0;i<NBUF;i++)
    {
        b=&bcache.buf[i];

        initsleeplock(&b->lock, "buffer");

        b->valid = 0;
        b->disk = 0;
        b->refcnt = 0;
        b->timestamp = 0;

        //将缓存块均匀分布到各个桶
        int h = i % NBUCKET;
        b->bucket = h;

        bininsert(&bcache.buckets[h], b);
    }
}

```

6. 修改 bget()

```

1  static struct buf*
2  bget(uint dev, uint blockno)
3  {
4      struct buf *b;
5      struct buf *victim;
6      int h;
7
8      h=bhash(blockno);//计算目标桶
9
10     //目标桶里查找
11     acquire(&bcache.buckets[h].lock);
12
13     // Is the block already cached?
14     //循环双向链表
15     for(b = bcache.buckets[h].head.next; b != &bcache.buckets[h].head; b = b->next){

```

```

16     if(b->dev == dev && b->blockno == blockno){
17         b->refcnt++;
18         release(&bcache.buckets[h].lock);
19         acquiresleep(&b->lock);
20         return b;
21     }
22 }
23
24 release(&bcache.buckets[h].lock);
25
26 // Not cached.
27 // Recycle the least recently used (LRU) unused buffer.
28 //缓存未命中时获取全局锁
29 //保证同一时刻只有一个线程执行缓存淘汰
30 acquire(&bcache.lock);
31
32 //再次检查，防止第一次查找失败到获取 bcache.lock 之间，其他线程已经把这个磁盘块加入缓存
33
34 acquire(&bcache.buckets[h].lock);
35
36 for (b = bcache.buckets[h].head.next; b != &bcache.buckets[h].head; b = b->next) {
37     if (b->dev == dev && b->blockno == blockno){
38         b->refcnt++;
39         release(&bcache.buckets[h].lock);
40         release(&bcache.lock);
41
42         acquiresleep(&b->lock);
43         return b;
44     }
45 }
46
47 release(&bcache.buckets[h].lock);
48
49 //选择 refcnt == 0 且 timestamp 最小的缓存块。
50
51 retry_victim:
52
53 victim = 0;
54 uint oldest = (uint)-1;
55
56 for (int i = 0; i < NBUF; i++)
57 {
58     b = &bcache.buf[i];
59
60     int bh = b->bucket;
61
62     acquire(&bcache.buckets[bh].lock);
63
64     if (b->refcnt == 0 && (victim == 0 || b->timestamp < oldest)) {
65         victim = b;
66         oldest = b->timestamp;
67     }
68     release(&bcache.buckets[bh].lock);

```

```

69     }
70
71     //都有进程
72     if (victim == 0)
73     {
74         release(&bcache.lock);
75         panic("bget: no buffers");
76     }
77
78     //扫描期间普通的缓存命中仍可增加 victim->refcnt, 重新获取 victim 所在桶的锁, 并再次检查
79     int oldh = victim->bucket;
80
81     acquire(&bcache.buckets[oldh].lock);
82
83     //重新检查 victim 是否仍然空闲
84     if (victim->refcnt != 0) {
85         release(&bcache.buckets[oldh].lock);
86         goto retry_victim;
87     }
88
89     if (oldh == h) //旧块和新块映射到同一个桶, 直接修改其身份
90     {
91         victim->dev = dev;
92         victim->blockno = blockno;
93         victim->valid = 0;
94         victim->refcnt = 1;
95         victim->timestamp = ticks;
96
97         release(&bcache.buckets[oldh].lock);
98     }
99     else //victim要从旧桶移动到新桶
100     {
101         //当前持有旧桶锁。由于 bcache.lock 保证只有一个淘汰线程, 而普通命中只会获取一把桶锁, 所以这里获
102         //取新桶锁不会与另一个淘汰线程形成锁环。
103         acquire(&bcache.buckets[h].lock);
104
105         bremove(victim);
106
107         victim->dev = dev;
108         victim->blockno = blockno;
109         victim->valid = 0;
110         victim->refcnt = 1;
111         victim->timestamp = ticks;
112         victim->bucket = h;
113
114         binset(&bcache.buckets[h], victim);
115
116         release(&bcache.buckets[h].lock);
117         release(&bcache.buckets[oldh].lock);
118     }
119
120     release(&bcache.lock);

```

```

121 //缓冲区缓存为每个缓冲区使用一把睡眠锁，确保任意时刻只有一条线程使用该缓冲区
122 acquiresleep(&victim->lock);
123 return victim;
124 }

```

7. 修改 `brelease()`

将原来的全局 LRU 链表操作改为只更新当前桶中的引用计数，并且当引用计数变为0时，记录释放时间，供之后选择最久未使用的缓冲区。

```

1 void
2 brelease(struct buf *b)
3 {
4     if(!holdingsleep(&b->lock))
5         panic("brelease");
6
7     releasesleep(&b->lock);
8
9     int h = b->bucket;
10
11     acquire(&bcache.buckets[h].lock);
12     b->refcnt--;
13
14     if (b->refcnt == 0) {
15         b->timestamp=ticks;
16     }
17     release(&bcache.buckets[h].lock);
18 }

```

8. 修改 `bpin()`

```

1 void
2 bpin(struct buf *b){
3     int h = b->bucket;
4     acquire(&bcache.bucket[h].lock);
5     b->refcnt++;
6     release(&bcache.bucket[h].lock);
7 }

```

9. 修改 `bunpin()`

```

1 void
2 bunpin(struct buf *b){
3     int h = b->bucket;
4
5     acquire(&bcache.bucket[h].lock);
6     if (b->refcnt < 1)
7         panic("bunpin");
8     b->refcnt--;
9     if (b->refcnt == 0)
10        b->timestamp = ticks;
11
12    release(&bcache.bucket[h].lock);
13 }

```

Lab总结

实验结果:

```

== Test running kalloc test ==
$ make qemu-gdb
(42.2s)
== Test    kalloc test: test1 ==
    kalloc test: test1: OK
== Test    kalloc test: test2 ==
    kalloc test: test2: OK
== Test kalloc test: sbrkmuch ==
$ make qemu-gdb
kalloc test: sbrkmuch: OK (4.5s)
== Test running bcachetest ==
$ make qemu-gdb
(3.8s)
== Test    bcachetest: test0 ==
    bcachetest: test0: OK
== Test    bcachetest: test1 ==
    bcachetest: test1: OK
== Test usertests ==
$ make qemu-gdb
usertests: OK (62.1s)
== Test time ==
time: OK
Score: 70/70

```

问题与解决

1. 可能为同一磁盘块建立多个缓存副本

两个进程可能同时第一次查找某个块，并且都发现缓存未命中。如果它们分别选择空闲缓冲区，就可能为相同的 `(dev, blockno)` 创建两个缓存副本，破坏块缓存的唯一性。为了避免发生这个问题，缓存未命中后先获取全局淘汰锁，然后再次查找目标哈希桶。只有第二次查找仍然失败时，才选择空闲缓冲区进行替换。全局锁保证同一时间只有一个线程执行缓存淘汰和新块创建。

2. 发生重复加锁

被替换缓冲区原来所在的桶可能与新块所在的桶相同。如果不区分这种情况，可能对同一把自旋锁重复执行 `acquire()`，从而触发错误。为了避免发生这种情况，分别处理 `oldh == h` 和 `oldh != h`：

- 如果新旧桶相同，直接在持有当前桶锁时修改缓存块信息；
- 如果新旧桶不同，则在全局淘汰锁的保护下完成删除、修改和插入操作。

实验心得

本次实验让我进一步理解了锁不仅用于保证程序正确性，也会直接影响多核程序的并行性能。即使程序不存在数据错误，如果大量操作都依赖同一把锁，多核处理器也会因为锁竞争退化为近似串行执行。

在物理内存分配器中，将一条全局空闲链表拆分为多条每 CPU 链表，本质上是通过数据分区减少共享状态。大多数内存操作只访问该 CPU 的链表，只有资源不足时才需要与其他 CPU 协调。这种方法能够兼顾性能和资源利用率。块缓存的实现更加复杂，因为磁盘块是真正需要在不同 CPU 和进程之间共享的数据，不能简单地为每个 CPU 保存独立副本。

实验还让我认识到，减小锁粒度会明显增加并发控制的复杂度。必须仔细考虑各种可能情况，避免出现死锁。

Lab: file system

Large files

要求：

题目要求扩展 xv6 文件系统的最大文件容量，在 inode 中保留 11 个直接块地址，并使用一个一级间接块和一个二级间接块。需要修改 inode 地址结构、`bmap()` 的块映射与按需分配逻辑，以及 `itrunc()` 的完整释放逻辑，同时保持磁盘 inode 大小不变，并确保内存 inode 与磁盘 inode 的地址数组一致。最终使 `bigfile` 能写入 65803 个数据块。

解决：

1. 切换到 `fs` 分支

```
1 | git checkout fs
```

2. 修改直接块和间接块数量

在 `kernel/fs.h` 中进行修改：

```
1 | #define NDIRECT 11
2 | #define NINDIRECT (BSIZE / sizeof(uint))
3 | #define NDINDIRECT (NINDIRECT * NINDIRECT)
4 | #define MAXFILE (NDIRECT + NINDIRECT + NDINDIRECT)
```

这样，最大文件块数会变为：

```
1 | 11 + 256 + 256×256 = 65803
```

3. 修改磁盘 inode 地址数组

在 `kernel/fs.h` 的 `struct dinode` 中修改：

```
1 | uint addrs[NDIRECT + 2];
```

4. 修改内存 inode 地址数组

在 kernel/file.h 的 struct inode 中修改:

```
1 | uint addrs[NDIRECT + 2];
```

让内存 inode 和磁盘 inode 的地址数组长度保持一致。

5. 修改 bmap() 的处理

kernel/fs.c 的 bmap() 作用是把文件内部的“逻辑块号”转换成磁盘上的“物理块号”；如果对应磁盘块还不存在，就分配一个新块。

修改逻辑为：

```
1 | static uint
2 | bmap(struct inode *ip, uint bn)
3 | {
4 |     uint addr,*a;
5 |     struct buf *bp;
6 |     //直接块
7 |     if(bn<NDIRECT){
8 |         if((addr=ip->addrs[bn])==0)
9 |             ip->addrs[bn]=addr=balloc(ip->dev); //在ip所属的磁盘上找一个空闲块，将其标记为已使用并清
           零，然后返回这个块的物理编号。
10 |         return addr;
11 |     }
12 |
13 |     bn=bn-NDIRECT;
14 |
15 |     //一级间接块
16 |     if(bn<NINDIRECT){
17 |         if((addr=ip->addrs[NDIRECT])==0)
18 |             ip->addrs[NDIRECT]=addr=balloc(ip->dev);
19 |
20 |         //把指定磁盘块读取到内存的缓冲区
21 |         bp=bread(ip->dev,addr);
22 |         //块内容看作地址数组
23 |         a = (uint*)bp->data;
24 |
25 |         // 如果对应的数据块不存在，就分配数据块
26 |         if((addr = a[bn]) == 0){
27 |             a[bn] = addr = balloc(ip->dev);
28 |             log_write(bp); //告诉xv6日志系统，缓冲区bp已被修改，需要作为当前文件系统事务的一部分写回磁盘。
29 |         }
30 |
31 |         brelse(bp); //使用完释放，释放 bp 上的睡眠锁，减少这个缓冲区的引用计数 refcnt
32 |         return addr;
33 |     }
34 |
35 |     //去掉一级间接块占用的逻辑块号范围
36 |     bn =bn-NINDIRECT;
37 |
38 |     if(bn<NDINDIRECT)
39 |     {
```

```

40     uint first;
41     uint second;
42
43     first=bn/NINDIRECT;
44     second=bn%NINDIRECT;
45
46     //分配最外层的二级间接块
47     if((addr = ip->addrs[NDIRECT + 1]) == 0)
48         ip->addrs[NDIRECT + 1] = addr = balloc(ip->dev);
49
50     //读取二级间接块
51     bp = bread(ip->dev, addr);
52     a = (uint*)bp->data;
53
54     // 分配二级间接块所指向的一级间接块
55     if((addr = a[first]) == 0){
56         a[first] = addr = balloc(ip->dev);
57         log_write(bp);
58     }
59
60     brelse(bp);
61
62     // 读取刚才找到或创建的一级间接块
63     bp = bread(ip->dev, addr);
64     a = (uint*)bp->data;
65
66     // 分配真正的数据块
67     if((addr = a[second]) == 0){
68         a[second] = addr = balloc(ip->dev);
69         log_write(bp);
70     }
71     brelse(bp);
72     return addr;
73 }
74 panic("bmap: out of range");
75 }

```

6. 修改 `itrunc()` 释放块

`kernel/fs.c` 的 `itrunc()` 作用是清空一个 `inode` 对应文件的全部内容，释放文件占用的所有数据块，并把文件大小设为 0。需要释放直接块，一级间接块和二级间接块。

增加：

```

}

if(ip->addr[NDIRECT+1]){
    bp = bread(ip->dev, ip->addr[NDIRECT+1]);
    a = (uint*)bp->data;
    for(i = 0; i < NINDIRECT; i++){
        if(a[i])
        {
            struct buf *bp2;
            uint *a2;
            bp2=bread(ip->dev,a[i]);
            a2=(uint*)bp2->data;

            for(j = 0; j < NINDIRECT; j++){
                if(a[j])
                    bfree(ip->dev, a2[j]);
            }

            brelse(bp2);

            bfree(ip->dev, a[i]);
        }
    }
    brelse(bp);

    bfree(ip->dev, ip->addr[NDIRECT+1]);
    ip->addr[NDIRECT+1] = 0;
}

```

Symbolic links

要求:

为 xv6 添加符号链接功能, 实现 `symlink(target, path)` 系统调用, 创建保存目标路径的新型 inode, 并在 `open()` 时自动递归解析符号链接。增加 `T_SYMLINK` 文件类型和 `O_NOFOLLOW` 标志, 使程序可以选择打开链接本身, 同时设置最大解析深度以检测循环链接。其他系统调用仍操作符号链接本身, 不需要支持目录符号链接。

解决:

1. 给 `symlink` 分配系统调用编号

在 `kernel/syscall.h` 中添加一个编号:

```
1 | #define SYS_symlink 22
```

2. 在用户态声明 `symlink()`

在 `user/user.h` 的系统调用声明区域添加:

```
1 | int symlink(const char*,const char*); //target path
```

创建符号链接 `path`, 使其指向 `target`。

3. 在 `user/usys.pl` 中添加系统调用桩

添加:

```
1 | entry("symlink");
```

4. 在内核中注册系统调用

在 `kernel/syscall.c` 中添加声明:

```
1 | extern uint64 sys_symlink(void);
```

然后在 `syscalls[]` 数组中添加:

```
1 | [SYS_symlink] sys_symlink,
```

5. 添加符号链接文件类型

在 `kernel/stat.h` 中添加:

```
1 | #define T_SYMLINK 4 //符号链接
```

6. 添加 `O_NOFOLLOW` 标志

在 `kernel/fcntl.h` 中添加:

```
1 | #define O_NOFOLLOW 0x800 //打开软链接本身，按位或组合
```

7. 把测试程序加入编译

在 `Makefile` 的 `UPROGS` 下添加:

```
1 | $U/_symlinktest\
```

8. 实现 `sys_symlink()`

在 `kernel/sysfile.c` 中添加:

```
1 | //接收用户传入的目标路径 target 和链接路径 path，创建一个类型为 T_SYMLINK 的 inode，并把目标路径字
   | 符串保存为这个 inode 的文件内容。
2 | uint64 sys_symlink(void){
3 |     char target[MAXPATH];
4 |     char path[MAXPATH];
5 |     struct inode *ip;
6 |     int n;
7 |
8 |     // 读取用户传入的两个字符串参数
9 |     if(argstr(0, target, MAXPATH) < 0 || argstr(1, path, MAXPATH) < 0)
10 |         return -1;
11 |
12 |     begin_op();
13 |
14 |     // 创建类型为 T_SYMLINK 的 inode
```

```

15     if((ip = create(path, T_SYMLINK, 0, 0)) == 0){
16         end_op();
17         return -1;
18     }
19
20     // 把结尾的 '\0' 也写入 inode, 便于之后直接作为路径使用
21     n = strlen(target) + 1;
22
23     if(writei(ip, 0, (uint64)target, 0, n) != n){
24         iunlockput(ip);
25         end_op();
26         return -1;
27     }
28
29     iunlockput(ip);
30     end_op();
31
32     return 0;
33 }

```

9. 在 `open()` 中跟随符号链接

在 `kernel/sysfile.c` 的 `sys_open()` 中添加:

```

1  // 如果没有设置 O_NOFOLLOW, 就不断跟随符号链接
2  if(!(omode & O_NOFOLLOW)){
3      int depth = 0;
4
5      while(ip->type == T_SYMLINK){
6          char target[MAXPATH];
7
8          // 防止 a -> b -> a 之类的循环
9          if(depth++ >= 10){
10             iunlockput(ip);
11             end_op();
12             return -1;
13         }
14
15         n = readi(ip, 0, (uint64)target, 0, MAXPATH);
16
17         if(n <= 0 || n > MAXPATH){
18             iunlockput(ip);
19             end_op();
20             return -1;
21         }
22
23         target[MAXPATH - 1] = '\0';
24
25         // 释放当前符号链接 inode
26         iunlockput(ip);
27
28         // 查找符号链接指向的路径
29         if((ip = namei(target)) == 0){

```

```

30     end_op();
31     return -1;
32 }
33 //锁定新目标inode
34 ilock(ip);
35 }
36 }

```

Lab总结

实验结果:

```

== Test running bigfile ==
$ make qemu-gdb
running bigfile: OK (105.9s)
== Test running symlinktest ==
$ make qemu-gdb
(0.4s)
== Test  symlinktest: symlinks ==
symlinktest: symlinks: OK
== Test  symlinktest: concurrent symlinks ==
symlinktest: concurrent symlinks: OK
== Test usertests ==
$ make qemu-gdb
usertests: OK (157.2s)
== Test time ==
time: OK
Score: 100/100

```

问题与解决

1. 新分配的间接块信息没有持久化

在 `bmap()` 中修改间接块中的地址后, 没有调用 `log_write()`, 更新可能无法正确写入日志和磁盘。解决这个问题, 每次向间接块写入新块号后调用:

```

1 log_write(bp);

```

2. 不理解符号连接的含义

最初不理解题目的要求, 不清楚符号链接保存的究竟是目标文件的 inode, 还是目标文件的路径。

通过分析查阅, 明确软连接本身是一个独立的特殊文件, 其 inode 类型为 `T_SYMLINK`, 数据块中保存的是目标文件的路径字符串, 而不是目标文件的 inode 编号。

例如:

```

1 symlink("/a/file", "/link");

```

表示创建一个名为 `/link` 的符号链接, 其内容为:

```

1 /a/file

```

访问:

```
1 | open("/link", O_RDONLY);
```

时，内核先找到 `/link` 的 inode，发现其类型为 `T_SYMLINK`，然后读出保存的路径 `/a/file`，再通过 `namei()` 查找真正的目标文件。

实验心得

通过 Large files 实验，我进一步理解了 xv6 inode 从逻辑块号到磁盘块号的映射过程。二级间接块并不是简单增加一个地址，而是增加了一层索引结构。实现时不仅要考虑文件增长时如何分配，还必须考虑文件删除时如何沿相反方向完整释放资源。这让我认识到，修改数据结构后，分配、访问、持久化和释放逻辑必须保持一致。Symbolic links 实验让我认识了符号链接。符号链接的本质不是直接保存目标 inode，而是保存目标路径，因此能够建立指向尚不存在文件的链接。

总而言之，这两个实验使我进一步熟悉了 xv6 的 inode 结构、路径查找等内容。同时认识到，修改文件系统不能只考虑正常访问，还必须同步处理资源释放、磁盘格式一致性问题，才能保证系统稳定运行。

Lab: mmap

Lab: mmap

要求：

为 xv6 实现文件内存映射系统调用 `mmap()` 和 `munmap()`。使用 VMA 记录每个进程的映射区域，并通过缺页异常按需分配页面、读取文件内容和设置访问权限；解除 `MAP_SHARED` 映射时需将修改写回文件。同时处理部分解除映射、进程退出清理及 `fork()` 继承映射，最终通过 `mmaptest` 和 `usertests`。

也就是为 xv6 添加文件内存映射功能，让程序可以像访问普通内存一样读取或修改文件内容，并能在不需要时取消映射。内核需要负责记录映射区域、按需加载文件数据、保存共享映射的修改，以及在进程退出或创建子进程时正确处理这些映射。

解决方法：

1. 切换到 `mmap` 分支

```
1 | git checkout mmap
```

2. 将测试程序加入编译

在 `Makefile` 的 `UPROGS` 中添加：

```
1 | $U/_mmaptest\
```

3. 在用户态声明系统调用

在 `user/user.h` 中添加：

```
1 | void *mmap(void *, int, int, int, int, int);
2 | int munmap(void *, int);
```

4. 添加用户态系统调用桩

在 `user/usys.pl` 中添加:

```
1 entry("mmap");
2 entry("munmap");
```

5. 分配系统调用编号

在 `kernel/syscall.h` 中添加编号, 例如:

```
1 #define SYS_mmap 22
2 #define SYS_munmap 23
```

6. 在内核中注册系统调用

在 `kernel/syscall.c` 中添加:

```
1 extern uint64 sys_mmap(void);
2 extern uint64 sys_munmap(void);
```

在 `syscalls[]` 数组中添加:

```
1 [SYS_mmap] sys_mmap,
2 [SYS_munmap] sys_munmap,
```

7. 为每个进程添加 VMA

在 `kernel/proc.h` 的 `struct proc` 前添加:

```
1 #define NVMA 16
2
3 struct vma {
4     int used;           // 这个 VMA 是否正在使用
5     uint64 addr;        // 映射起始虚拟地址
6     uint64 length;      // 实际请求的映射长度
7     int prot;           // PROT_READ、PROT_WRITE
8     int flags;          // MAP_SHARED、MAP_PRIVATE
9     uint64 offset;      // 文件起始偏移
10    struct file *file;   // 被映射的文件
11 };
```

然后在 `struct proc` 中添加:

```
1 struct vma vmas[NVMA]; // 虚拟内存区域
```

8. 初始化进程的 VMA

在 `kernel/proc.c` 的 `allocproc()` 中添加:

```
1 // 初始化清零
2 memset(p->vmas, 0, sizeof(p->vmas));
```

9. 实现 `sys_mmap()`

在 `kernel/sysfile.c` 中添加:

```
1  uint64 sys_mmap(void)
2  {
3      uint64 requested_addr;
4      int length;
5      int prot;
6      int flags;
7      int fd;
8      int offset;
9      struct file *f;
10     struct proc *p = myproc();
11     struct vma *v = 0;
12
13     if(argaddr(0, &requested_addr) < 0)
14         return -1;
15     if(argint(1, &length) < 0)
16         return -1;
17     if(argint(2, &prot) < 0)
18         return -1;
19     if(argint(3, &flags) < 0)
20         return -1;
21     if(argfd(4, &fd, &f) < 0)
22         return -1;
23     if(argint(5, &offset) < 0)
24         return -1;
25
26     // MAP_SHARED + PROT_WRITE 最终需要写回文件,
27     // 因此文件必须以可写方式打开。
28     if((flags & MAP_SHARED) && (prot & PROT_WRITE) && f->writable == 0)
29         return -1;
30
31     // 找一个空闲 VMA。
32     for(int i = 0; i < NVMA; i++){
33         if(p->vmass[i].used == 0){
34             v = &p->vmass[i];
35             break;
36         }
37     }
38
39     if(v == 0)
40         return -1;
41
42     uint64 maplen = PGROUNDUP((uint64)length);
43
44     /*
45      * TRAMPOLINE 位于 MAXVA-PGSIZE,
46      * trapframe 位于 MAXVA-2*PGSIZE。
47      * mmap 区域从 trapframe 下方向低地址增长。
48      */
49     uint64 addr = MAXVA - 2 * PGSIZE;
```

```

50
51     for(int i = 0; i < NVMA; i++) {
52         if(p->vmass[i].used && p->vmass[i].addr < addr)
53             addr = p->vmass[i].addr;
54     }
55
56     //防止无符号减法下溢
57     if(addr < maplen)
58         return -1;
59
60     //在最低 VMA 的下面分配
61     addr -= maplen;
62
63     if(addr < PGROUNDUP(p->sz))
64         return -1;
65
66     v->used = 1;
67     v->addr = addr;
68     v->length = length;
69     v->prot = prot;
70     v->flags = flags;
71     v->offset = offset;
72     v->file = filedup(f);
73
74     return addr;
75 }

```

这个函数的作用是：在进程的虚拟地址空间中建立一段“文件到内存”的映射，并把这段映射的起始虚拟地址返回给用户程序。这里只记录 VMA，不分配物理页，也不读取文件。

10. 实现 mmap 缺页加载

在 `kernel/trap.c` 中添加辅助函数，作用是处理进程第一次访问 `mmap()` 映射区域时产生的缺页异常，按需分配物理页，并把对应的文件内容加载到该页面中。

```

1  static int mmap_page_fault(struct proc *p, uint64 fault_addr, uint64 cause){
2      struct vma *v = 0;
3
4      // 找到包含缺页地址的 VMA
5      for(int i = 0; i < NVMA; i++) {
6          if(p->vmass[i].used && fault_addr >= p->vmass[i].addr && fault_addr < p->vmass[i].addr +
p->vmass[i].length)
7              {
8                  v = &p->vmass[i];
9                  break;
10             }
11     }
12
13     if(v == 0)
14         return -1;
15
16     // load page fault, 但 VMA 不允许读
17     if(cause == 13 && !(v->prot & PROT_READ))

```

```

18     return -1;
19
20     // store page fault, 但 VMA 不允许写
21     if(cause == 15 && !(v->prot & PROT_WRITE))
22         return -1;
23
24     // 映射必须以页为单位
25     uint64 va = PGROUNDDOWN(fault_addr);
26
27     char *mem = kalloc();
28     if(mem == 0)
29         return -1;
30
31     // 文件最后一页可能不足 4096 字节, 剩余部分应为 0
32     memset(mem, 0, PGSIZE);
33
34     uint64 file_offset = v->offset + (va - v->addr);
35     uint64 remaining = v->length - (va - v->addr);
36     uint64 read_size = remaining < PGSIZE ? remaining : PGSIZE;
37
38     // 从文件中读取这一页对应的数据
39     ilock(v->file->ip);
40     int n = readi(v->file->ip, 0, (uint64)mem, file_offset, read_size);
41     iunlock(v->file->ip);
42
43     if(n < 0) {
44         kfree(mem);
45         return -1;
46     }
47
48     //构造页表权限
49     int perm = PTE_U;
50
51     if(v->prot & PROT_READ)
52         perm |= PTE_R;
53
54     //可写页面/同时添加 PTE_R
55     if(v->prot & PROT_WRITE)
56         perm |= PTE_R | PTE_W;
57
58     // 建立用户虚拟地址到新物理页的映射
59     if(mappages(p->pagetable, va, PGSIZE, (uint64)mem, perm) != 0){
60         kfree(mem);
61         return -1;
62     }
63     return 0;
64 }

```

11. 在 usertrap() 中处理 mmap 缺页

在 kernel/trap.c 的 usertrap() 中添加:

```

1  else if(r_scause() == 13 || r_scause() == 15)
2  {
3      //13独缺页 15写缺页
4      if(mmap_page_fault(p, r_stval(), r_scause()) < 0)
5          p->killed = 1;
6  }

```

12. 声明 VMA 解除映射函数

在 `kernel/defs.h` 中添加:

```

1  int vma_munmap(struct proc *, uint64, uint64); //解除映射指定的页面

```

供 `sys_munmap()`、`exit()` 和 `exec()` 共同调用。

13. 实现共享页面写回函数

在 `kernel/proc.c` 中添加:

```

1  //写回函数
2  static int vma_write_page(struct vma *v, uint64 va, uint64 pa, uint64 n)
3  {
4      uint64 done = 0;
5      uint64 file_offset = v->offset + (va - v->addr);
6
7      //使用与 fwrite 类似的方法
8      int max = ((MAXOPBLOCKS - 1 - 1 - 2) / 2) * BSIZE;
9
10     while(done < n) {
11         uint64 count = n - done;
12         if(count > max)
13             count = max;
14
15         begin_op();
16         ilock(v->file->ip);
17         int written = writei(v->file->ip, 0, pa + done, file_offset + done, count);
18         iunlock(v->file->ip);
19         end_op();
20
21         if(written != count)
22             return -1;
23
24         done += written;
25     }
26     return 0;
27 }

```

14. 实现 `vma_munmap()`

继续在 `kernel/proc.c` 中添加:

```

1  int vma_munmap(struct proc *p, uint64 addr, uint64 length)

```

```

2 {
3     struct vma *v = 0;
4
5     if(length == 0)
6         return -1;
7
8     for(int i = 0; i < NVMA; i++) {
9         if(p->vmass[i].used && addr >= p->vmass[i].addr && addr + length <= p->vmass[i].addr +
p->vmass[i].length) {
10             v = &p->vmass[i];
11             break;
12         }
13     }
14
15     if(v == 0)
16         return -1;
17
18     uint64 old_addr = v->addr;
19     uint64 old_length = v->length;
20     uint64 end = addr + length;
21
22     //只会删除开头、结尾或者整个 VMA
23     if(addr != old_addr && end != old_addr + old_length)
24         return -1;
25
26     uint64 first = PGROUNDNDOWN(addr);
27     uint64 last = PGROUNDUP(end);
28
29     for(uint64 va = first; va < last; va += PGSIZE)
30     {
31         pte_t *pte = walk(p->pagetable, va, 0);
32
33         if(pte == 0 || (*pte & PTE_V) == 0)
34             continue;
35
36         if(v->flags & MAP_SHARED)
37         {
38             uint64 page_start = va;
39             uint64 page_end = va + PGSIZE;
40             uint64 mapped_end = old_addr + old_length;
41
42             if(page_end > mapped_end)
43                 page_end = mapped_end;
44
45             if(page_end > page_start)
46             {
47                 uint64 pa = PTE2PA(*pte);
48                 uint64 n = page_end - page_start;
49
50                 if(vma_write_page(v, va, pa, n) < 0)
51                     return -1;
52             }
53         }
54     }
55 }

```

```

54     uvmunmap(p->pagetable, va, 1, 1);
55 }
56 //整个
57 if(addr == old_addr && end == old_addr + old_length)
58 {
59     fclose(v->file);
60     memset(v, 0, sizeof(*v));
61 } else if(addr == old_addr)//开头
62 {
63     v->addr += length;
64     v->offset += length;
65     v->length -= length;
66 } else //结尾
67 {
68     v->length -= length;
69 }
70 return 0;
71 }

```

该函数作用是在当前进程的虚拟地址空间中，解除从 `addr` 开始、长度为 `length` 字节的 `mmap` 映射。

15. 实现 `sys_munmap()`

在 `kernel/sysfile.c` 中添加:

```

1  uint64 sys_munmap(void)
2  {
3      uint64 addr;
4      int length;
5
6      if(argaddr(0, &addr) < 0 || argint(1, &length) < 0)
7          return -1;
8
9      if(length <= 0)
10         return -1;
11
12     return vma_munmap(myproc(), addr, length);
13 }

```

16. 在 `exit()` 中清理全部 VMA

在 `kernel/proc.c` 的 `exit()` 中:

```

1  //解除映射
2  for(int i = 0; i < NVMA; i++) {
3      if(p->vmass[i].used) {
4          uint64 addr = p->vmass[i].addr;
5          uint64 length = p->vmass[i].length;
6          vma_munmap(p, addr, length);
7      }
8  }

```

17. 让 `fork()` 继承 VMA

在 `kernel/proc.c` 的 `fork()` 中添加:

```
1  for (i = 0; i < NVMA; i++) {
2      if (p->vmass[i].used) {
3          np->vmass[i] = p->vmass[i];
4
5          np->vmass[i].file =
6              filedup(p->vmass[i].file);
7      }
8  }
```

子进程只继承 VMA 信息，不共享父进程的 `mmap` 物理页。子进程首次访问时重新从文件加载页面。

18. 在 `exec()` 中清理旧映射

在 `kernel/exec.c` 中添加:

```
1  for (int i = 0; i < NVMA; i++) {
2      if (p->vmass[i].used) {
3          uint64 addr = p->vmass[i].addr;
4          uint64 length = p->vmass[i].length;
5
6          vma_munmap(p, addr, length);
7      }
8  }
```

Lab总结

实验结果:


```

== Test running mmaptest ==
$ make qemu-gdb
(3.1s)
== Test    mmaptest: mmap f ==
    mmaptest: mmap f: OK
== Test    mmaptest: mmap private ==
    mmaptest: mmap private: OK
== Test    mmaptest: mmap read-only ==
    mmaptest: mmap read-only: OK
== Test    mmaptest: mmap read/write ==
    mmaptest: mmap read/write: OK
== Test    mmaptest: mmap dirty ==
    mmaptest: mmap dirty: OK
== Test    mmaptest: not-mapped unmap ==
    mmaptest: not-mapped unmap: OK
== Test    mmaptest: two files ==
    mmaptest: two files: OK
== Test    mmaptest: fork_test ==
    mmaptest: fork_test: OK
== Test usertests ==
$ make qemu-gdb
usertests: OK (55.1s)
== Test time ==
time: OK
Score: 140/140

```

问题与解决

在最初实现 `munmap()` 时，只使用 `uvmunmap()` 解除了用户页表中的映射并释放物理页，没有处理映射页面中被修改的数据。因此，程序通过映射内存修改文件内容后，内存中的内容虽然发生了变化，但磁盘文件仍然保持原来的数据。

问题的原因是，缺页处理程序使用 `readi()` 将文件内容读取到新分配的物理页后，文件和物理页只是内容相同，两者之间并不存在自动同步关系。用户修改映射页面时，实际修改的是物理内存，不会自动调用文件系统的写操作。为了解决这一问题，在 `munmap()` 中判断映射类型；对于 `MAP_SHARED`，先通过 `writel()` 将页面内容写回文件，再调用 `uvmunmap()` 解除映射。对于 `MAP_PRIVATE`，直接释放页面。

实验心得

本实验让我进一步理解了虚拟内存、文件系统和缺页异常之间的联系。`mmap()` 并不是简单地将文件内容复制到内存，而是先用 VMA 描述一段虚拟地址范围，在程序真正访问时再通过缺页异常分配页面并读取文件，这体现了按需分配和懒加载的设计思想。

实验中最重要的是维护不同内核对象的生命周期，包括 VMA、页表映射、物理页和文件引用。任何一个环节处理不完整，都可能造成页面泄漏、文件引用泄漏或重复释放。

Lab: networking

Lab: networking

要求：

为 xv6 的 E1000 网卡实现设备驱动，完成 `e1000_transmit()` 和 `e1000_recv()`。通过 DMA 描述符环管理数据包发送与接收，正确更新 `E1000_TDT`、`E1000_RDT` 寄存器，分配和释放 `mbuf`，并使用锁保证并发安全。最终使 xv6 能够完成 ARP、UDP、ping 和 DNS 通信，并通过 `nettests` 与 `make grade`。

简而言之，就是为 xv6 编写一个简单的网卡驱动，让操作系统能够通过虚拟网卡发送和接收网络数据。程序需要管理待发送和已接收的数据包，并把网卡收到的数据交给 xv6 的网络协议栈处理，从而实现 xv6 与宿主机及互联网之间的基本通信。

解决：

1. 切换到 `net` 分支

```
1 | git checkout net
```

2. 实现 `e1000_transmit()`

在 `kernel/e1000.c` 中完成发送函数：

```
1 | int
2 | e1000_transmit(struct mbuf *m)
3 | {
4 |     acquire(&e1000_lock);
5 |
6 |     // TDT 指向驱动应该填写的下一个发送描述符。
7 |     uint32 index = regs[E1000_TDT]; // 读取网卡的 TDT 硬件寄存器
8 |     struct tx_desc *desc = &tx_ring[index];
9 |
10 |    // DD 没有置位，说明网卡还没有使用完这个描述符。
11 |    // 此时不能覆盖它，否则会破坏尚未完成的发送操作。
12 |    if ((desc->status & E1000_TXD_STAT_DD) == 0) {
13 |        release(&e1000_lock);
14 |        return -1;
15 |    }
16 |
17 |    // 网卡已经发送完这个位置原来的数据。
18 |    // 因此现在可以释放原来由这个描述符引用的 mbuf。
19 |    if (tx_mbufs[index] != 0) {
20 |        mbuf_free(tx_mbufs[index]);
21 |        tx_mbufs[index] = 0;
22 |    }
23 |
24 |    // 让描述符指向当前要发送的数据包。
25 |    desc->addr = (uint64)m->head;
26 |    desc->length = m->len;
27 |
28 |    // EOP: 这个描述符包含数据包的最后一段。RS: 发送完成后，请网卡把 DD 状态写回来。
29 |    desc->cmd = E1000_TXD_CMD_EOP | E1000_TXD_CMD_RS;
30 |
31 |    // 清除旧状态，表示这个描述符现在交给网卡处理。
32 |    desc->status = 0;
33 |
34 |    // 保存 mbuf，等网卡发送完成后再释放。
35 |    tx_mbufs[index] = m;
36 |
37 |    // 保证描述符内容先写入内存，再通知网卡。
38 |    __sync_synchronize();
39 |}
```

```

40 // 推进发送环的尾指针，通知网卡有新包需要发送。
41 regs[E1000_TDT] = (index + 1) % TX_RING_SIZE;
42
43 release(&e1000_lock);
44 return 0;
45 }

```

3. 实现 e1000_recv()

在 kernel/e1000.c 中完成接收函数：

```

1  static void
2  e1000_recv(void)
3  {
4      while (1)
5      {
6          acquire(&e1000_lock);
7
8          // RDT 指向驱动最后处理完并归还给网卡的描述符。
9          // 所以下一个可能包含新数据包的位置是 RDT + 1。
10         uint32 index = (regs[E1000_RDT] + 1) % RX_RING_SIZE;
11         struct rx_desc *desc = &rx_ring[index];
12
13         // 没有 DD，说明网卡还没有在这个描述符中放入新数据。
14         if ((desc->status & E1000_RXD_STAT_DD) == 0) {
15             release(&e1000_lock);
16             break;
17         }
18
19         // 取出网卡刚刚写入数据的旧 mbuf。
20         struct mbuf *m = rx_mbufs[index];
21         m->len = desc->length;
22
23         // 当前 mbuf 将交给网络协议栈。
24         // 因此必须给网卡准备一个新的空 mbuf。
25         struct mbuf *new_m = mbufalloc(0);
26         if (new_m == 0)
27             panic("e1000_recv");
28
29         rx_mbufs[index] = new_m;
30         desc->addr = (uint64)new_m->head;
31
32         // 清除完成状态，让网卡以后可以再次使用这个描述符。
33         desc->status = 0;
34
35         // 确保新缓冲区地址和状态已经写入描述符。
36         __sync_synchronize();
37
38         // 把这个描述符归还给网卡。
39         regs[E1000_RDT] = index;
40
41         release(&e1000_lock);
42

```

```
43 // 交给 IP/ARP/UDP 网络协议栈处理
44 net_rx(m);
45 }
46 }
```

Lab总结

实验结果：

```
== Test running nettests ==
$ make qemu-gdb
(3.6s)
== Test  nettest: ping ==
nettest: ping: OK
== Test  nettest: single process ==
nettest: single process: OK
== Test  nettest: multi-process ==
nettest: multi-process: OK
== Test  nettest: DNS ==
nettest: DNS: OK
== Test time ==
time: OK
Score: 100/100
```

问题与解决

本次题目的最大问题就是对于题目的理解，通过查阅，理解了本实验的核心是通过发送环和接收环与网卡交换数据：

- 发送时，CPU 从 `E1000_TDT` 指向的位置取得发送描述符，将 `mbuf` 的地址和长度写入描述符，再移动 `TDT`，通知网卡处理该数据包。
- 接收时，网卡先把数据写入接收描述符对应的 `mbuf`，然后设置 `DD` 状态位。CPU 检查到该状态位后，将数据包交给网络协议栈，并为描述符补充一个新的缓冲区。
- 描述符数组是环形结构，到达数组末尾后需要通过取模运算回到开头。

其中最重要的是正确理解 `DD` 状态位和 `mbuf` 的所有权。发送描述符的 `DD` 位没有设置时，说明网卡仍在使用该描述符，此时不能覆盖描述符中的内容，也不能释放旧的 `mbuf`。只有网卡发送完成并设置 `DD` 后，驱动才能释放旧缓冲区并重新使用该描述符。接收过程中的缓冲区所有权也发生了转移：原来的 `mbuf` 在接收到数据后要交给 `net_rx()`，不能继续留在接收环中。因此必须先分配一个新的 `mbuf` 放入描述符，再把旧的 `mbuf` 传给网络协议栈。

实验心得

总体来说，这次实验把网络协议栈、内存管理、并发控制和硬件设备操作联系在了一起。完成实验后，我不仅理解了 E1000 网卡驱动的基本收发过程，还加深了我对设备驱动、环形队列以及并发控制的理解。

总结

通过完成 xv6 的11个实验，我对操作系统从用户程序到内核底层实现的完整工作流程有了更加系统和深入的认识。这些实验并不是彼此孤立的，而是从 UNIX 用户工具开始，逐步深入系统调用、虚拟内存、异常处理、进程管理、并发控制、文件系统和设备驱动，最终形成了一条较为完整的操作系统知识链。

在 `util` 实验中，我通过实现 `sleep`、`pingpong`、`primes`、`find` 和 `xargs`，熟悉了 xv6 用户程序的编写方式，并掌握了 `fork`、`pipe`、`read`、`write`、`exec` 和 `wait` 等基本 UNIX 系统调用。这部分实验让我直观理解了进程创建、进程间通信、文件描述符以及管道的工作方式。

在 `syscall` 实验中，我实现了系统调用跟踪和系统信息查询功能，了解了一个系统调用从用户态接口、汇编调用桩、系统调用编号，到内核处理函数的完整执行路径。

在 `pgtbl` 实验中，深入学习了 RISC-V 三级页表、虚拟地址转换和页表项权限。通过打印页表、为每个进程建立独立的内核页表，以及同步用户地址映射，认识内核如何管理用户和内核地址空间。

在 `traps` 实验中，我学习了系统调用、中断和异常如何统一通过 trap 机制进入内核。通过分析 RISC-V 汇编、实现内核调用栈回溯和用户级定时报警，我对寄存器保存、栈帧结构、trapframe，以及用户态和内核态之间的切换过程有了更清晰的认识。

在 `lazy` 实验中，我实现了用户堆内存的懒分配。该实验让我理解了操作系统可以先扩展进程的虚拟地址范围，在进程真正访问页面并触发缺页异常时，再分配物理内存。同时还需要处理非法地址、内存不足、负数 `sbrk()`、用户栈保护页以及 `fork()` 等特殊情况。

在 `cow` 实验中，我通过写时复制优化了 `fork()`。父子进程最初共享相同的物理页面，只有在发生写操作时才复制页面。实现过程中需要正确设置页表权限、处理写缺页异常，并使用引用计数管理共享物理页的生命周期。

在 `thread` 实验中，我完成了用户级线程上下文切换、基于线程的哈希表并行化以及可重复使用的屏障。通过保存和恢复寄存器，我理解了线程切换的底层原理；通过互斥锁和条件变量，我认识到并发程序不仅需要保证正确性，还需要尽量减小锁的粒度，从而提升并行性能。

在 `lock` 实验中，我将物理内存分配器和磁盘缓冲区缓存中的全局锁拆分为多把锁。实验表明，虽然使用一把大锁更容易保证正确性，但会造成严重的锁竞争；使用每 CPU 空闲链表、哈希桶锁等细粒度设计，可以让互不冲突的操作并行执行。

在 `fs` 实验中，我通过实现大文件和符号链接功能，掌握了 inode、数据块、直接块、间接块、目录项和路径解析的工作方式。尤其是双重间接块的实现，使我理解了文件系统如何通过多级索引支持更大的文件；符号链接则让我进一步理解了路径查找和文件类型处理。

在 `mmap` 实验中，我实现了文件到进程虚拟地址空间的映射，并结合缺页异常实现按需加载。该实验综合运用了虚拟内存、文件系统、进程管理和异常处理等知识，也让我理解了 `MAP_SHARED`、`MAP_PRIVATE`、权限检查、写回文件和解除映射的基本机制。

在 `net` 实验中，我完成了 E1000 网卡驱动的数据包发送和接收功能。通过操作发送描述符环、接收描述符环和设备寄存器，我了解了操作系统与硬件设备之间的交互方式。

总体而言，xv6 实验让我将操作系统课程中的抽象概念与实际代码联系起来。理解了进程、系统调用、虚拟内存、文件系统、锁和设备驱动等基本原理，也体会到了操作系统各模块之间紧密的依赖关系。